BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

CƠ SỞ TẠI TP. HỒ CHÍ MINH

KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành: Công nghệ kỹ thuật điện, điện tử (Hệ Đại học Chính quy)

ĐỀ TÀI: DUYỆT CÂY THEO MỨC VÀ XÁC ĐỊNH ĐỘ RỘNG TỪNG MỨC

Mã đề: DT084

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Lương Minh Trực
Mã số sinh viên	N24DCDT100
Lớp	D24CQDT01-N (Niên khóa 2024-2029)

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

CƠ SỞ TẠI TP. HỒ CHÍ MINH

KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành: Công nghệ kỹ thuật điện, điện tử (Hệ Đại học Chính quy)

ĐỀ TÀI: DUYỆT CÂY THEO MỨC VÀ XÁC ĐỊNH ĐỘ RỘNG TỪNG MỨC

Mã đề: DT084

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Lương Minh Trực
Mã số sinh viên	N24DCDT100
Lớp	D24CQDT01-N (Niên khóa 2024-2029)
Học phần và nhóm	INT13185 — Nhóm 03
Phiên bản báo cáo	Bản hoàn thiện nội dung ngày 28/10/2026

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2026

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

CƠ SỞ TẠI TP. HỒ CHÍ MINH

PHIẾU GIAO ĐỀ TÀI

DT084 INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Sinh viên Lương Minh Trực MSSV N24DCDT100

Lớp D24CQDT01-N

Duyệt cây theo mức và xác định độ rộng từng mức

Chương 7, 7.3.2 Duyệt theo mức; trang in 165; trang PDF 177

Đề bài

Viết chương trình duyệt cây nhị phân theo từng mức bằng hàng đợi. Sinh viên thấy mỗi nút cha được lấy trước các nút con ở mức tiếp theo, ghi số nút mỗi mức và xác định độ rộng lớn nhất. Gốc có mức không; cây rỗng không có mức nào để in.

Dữ liệu vào

Nhập hoặc khai báo các bộ (id, giá_trị, trái, phải), ID=1..n, 0 là con rỗng; $0 \le n \le 100$. Có một cây với gốc chỉ định. Khi đưa con vào hàng, đưa trái trước phải.

Kết quả ra

In dãy theo mức, từng nhóm cùng mức, độ rộng lớn nhất và mức đầu tiên đạt độ rộng đó. Cây rỗng rộng 0, mức rộng nhất không tồn tại.

Ví dụ 1

Dữ liệu vào	Kết quả chính
`goc=1; nut=[(1,10,2,3),(2,20,4,0),(3,30,0,0),(4,40,0,0)]`	`cac_muc=[[10],[20,30],[40]]; rong_nhat=2; muc=1`

Hai con của gốc cùng mức 1 được xử lý trước cháu 40 ở mức 2.

Ví dụ 2

Dữ liệu vào	Kết quả chính
`goc=1; nut=[(1,8,0,0)]`	`cac_muc=[[8]]; rong_nhat=1; muc=0`

Chỉ có gốc nên một mức duy nhất rộng một; queue trở rỗng sau khi lấy gốc.

DT084 HƯỚNG DẪN THỰC HIỆN

N24DCDT100 Lương Minh Trực

Thực hiện lần lượt năm bước

1. Đọc duyệt mức trong sách và ghi thứ tự đưa trái/phải vào queue trên cây mẫu. Trước khi lập trình, tự viết kết quả dự kiến của ví dụ thứ nhất bằng tay.

2. Dựng cây và queue chứa (gốc,0), xử lý cây rỗng trước khi đưa vào. Chạy riêng hàm này trước khi ghép vào chương trình chính.

3. Mỗi lần lấy, ghi nút vào nhóm mức và đưa hai con tồn tại kèm mức cộng một. In trạng thái sau mỗi bước trên ví dụ đã cho.

4. Tính độ rộng từng nhóm, giữ mức đầu tiên khi nhiều mức cùng độ rộng lớn nhất. Nếu kết quả khác dự kiến, xác định bước đầu tiên bị sai rồi sửa.

5. Chạy hai ví dụ và năm ca, so kết quả với sơ đồ cây đã vẽ. Nộp mã có chú thích các biến và bảng kết quả năm ca kiểm.

Các hàm nên viết

```
duyet_muc(g): dãy và các nhóm
do_rong(cac_muc): số nút từng mức
muc_rong_nhat(cac_muc): chọn mức
```

Năm ca cần kiểm tra

1. Cây rỗng → rộng 0, không mức.

2. Một nút → rộng 1, mức 0.

3. Gốc có hai lá → rộng 2, mức 1.

4. Dây phải bốn nút → mọi mức rộng 1, chọn mức 0.

5. Cây đầy 7 nút → độ rộng [1,2,4].

Phân tích và nhận xét

Đếm số lần nút vào/ra queue; mỗi nút đúng một lần nên O(n). Queue chứa nút đang chờ, O(n) xấu nhất; nhóm kết quả O(n). Thử 3,7,31 nút, so queue lớn nhất giữa cây dây và cây gần đầy.

Yêu cầu thêm sau khi phần cơ bản chạy đúng

Tùy chọn: in trung bình giá trị từng mức bằng số thực, xử lý cây rỗng rõ ràng.

Sản phẩm: mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

Ngày giao ...../...../..... Ngày nộp ...../...../.....

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên)
Lương Minh Trực	Hồ Nhựt Minh

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Học phần INT13185 — Nhóm 03. Đề tài DT084. Sinh viên Lương Minh Trực — N24DCDT100. Các cột điểm dành cho giảng viên; báo cáo chỉ dẫn vị trí bằng chứng.

Bảng P.1. Phiếu chấm theo rubric chung

Tiêu chí	**Trọng số**	**GV1 /10**	**GV2 /10**	**Quy đổi**
C1 Đặc tả bài toán, input/output	1,5			
C2 Biểu diễn dữ liệu và phần tự cài	2,0			
C3 Thuật toán, tính đúng và truy vết	2,5			
C4 Kiểm thử và độ phức tạp	2,0			
C5 Báo cáo, minh chứng và GitHub	1,0			
C6 Bảo vệ và giải thích	1,0			

Điểm quy đổi = điểm tiêu chí × trọng số / 10; tổng tối đa 10 điểm. Cộng trước rồi làm tròn một chữ số thập phân; hai người chấm lấy trung bình theo rubric.

Tổng điểm: ...../10. Nhận xét: .....

Người chấm 1: ..... Người chấm 2: .....

Bảng P.2. Vị trí đối chiếu sáu tiêu chí

Tiêu chí	**Nội dung và bằng chứng**
C1	Chương 1; phiếu giao; Bảng 1.2–1.3.
C2	Mục 2.2, 3.2; src/DeTai.jl; FIFO và validator tự cài.
C3	Mục 3.3–3.5; Bảng 3.3–3.4; lập luận bất biến.
C4	Mục 3.6, Chương 4; 13 ca; 139 phép kiểm; khảo sát 3/7/31 nút.
C5	Hình 4.1–4.11; repo, SHA C, permalink; Phụ lục A–D.
C6	Chưa bảo vệ; menu/nhập liệu phục vụ chuẩn bị. Giảng viên đánh giá trực tiếp.

LỜI CAM ĐOAN

Em chịu trách nhiệm về tính trung thực của dữ liệu, kết quả và minh chứng trong báo cáo. Các ảnh A01–A11 được chụp từ việc chạy chương trình và truy cập repo cá nhân; kết quả kỳ vọng được đối chiếu với sơ đồ cây, đặc tả và các phép kiểm đã công bố. Các nội dung tham khảo từ giáo trình, phiếu giao và tài liệu môn học được ghi nguồn trong phần tài liệu tham khảo.

Quá trình hoàn thiện có sử dụng ChatGPT/Codex hỗ trợ phân tích, chỉnh sửa mã, bổ sung kiểm thử và biên tập báo cáo. Việc khai báo công cụ hỗ trợ nhằm phân biệt phần tham khảo, hỗ trợ và phần kiểm tra thực tế; em không coi nội dung do công cụ tạo ra là bằng chứng đã tự chạy. Em đã chạy các ca minh họa, lưu ảnh cá nhân và chịu trách nhiệm tìm hiểu để giải thích chương trình khi bảo vệ.

Báo cáo không tự gán điểm hoặc kết quả bảo vệ. Các kết luận về chương trình giới hạn trong hợp đồng đầu vào, lập luận thuật toán và tập kiểm thử đã nêu; không khẳng định đã kiểm vét cạn mọi cây nhị phân.

LỜI CẢM ƠN

Em cảm ơn giảng viên Hồ Nhựt Minh đã cung cấp phiếu giao đề tài, giáo trình, khung báo cáo và rubric để em xác định rõ phạm vi cần thực hiện. Các yêu cầu về đặc tả, truy vết, kỳ vọng độc lập và khả năng chạy lại giúp em kiểm tra chương trình thay vì chỉ dựa vào một kết quả minh họa.

Qua đề tài, em củng cố cách biểu diễn cây bằng ID, sử dụng hàng đợi FIFO và phân tích số thao tác theo số nút. Em tiếp tục hoàn thiện khả năng giải thích mã, xử lý dữ liệu sai và trình bày bằng chứng trong quá trình học tập.

TÓM TẮT

Đề tài DT084 xây dựng chương trình duyệt cây nhị phân theo mức và xác định độ rộng từng mức. Đầu vào gồm tối đa 100 nút, mỗi nút là bộ (id, giá_trị, trái, phải), có gốc chỉ định và dùng 0 cho con rỗng. Chương trình duyệt trái trước phải trong từng mức, nhóm các nút có cùng độ sâu, tính độ rộng lớn nhất và chọn mức nhỏ nhất khi hòa.

Bản cài đặt chính sử dụng Julia 1.12.6, tự cài hàng đợi bằng hai mảng và hai chỉ số. Validator kiểm kiểu dữ liệu, miền chỉ số, ID trùng, bậc vào, gốc có cha, liên thông và chu trình trước khi duyệt. Toàn bộ xử lý có thời gian O(n) và bộ nhớ O(n) theo mô hình thao tác trên mỗi giá trị có chi phí hằng số. Phần mở rộng tính trung bình theo mức bằng Float64 và trả nothing khi một mức chứa chuỗi.

Bộ thuật toán có 38 nhóm ca T01–T38 với 121 phép kiểm đạt; bộ nhập liệu có 16 nhóm I01–I16 với 18 phép kiểm đạt. Khảo sát trên cây dây và cây đầy ở n=3,7,31 cho mỗi nút vào/ra FIFO chính đúng một lần. Báo cáo kèm bảng truy vết, 13 ca đối chiếu đại diện, 11 ảnh cá nhân và permalink mã tại commit C để kiểm lại. Giao diện HTML hỗ trợ mô phỏng trực quan, hoạt động độc lập với mã Julia.

Từ khóa: cây nhị phân; duyệt theo mức; FIFO; độ rộng; Julia; kiểm thử.

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 2
1.3.1. Miền và quy ước đầu vào 2
1.3.2. Kết quả và trường hợp đặc biệt 2
1.4. Cách thực hiện và cấu trúc báo cáo 3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4
2.1. Khái niệm và mô hình dữ liệu 4
2.2. Cấu trúc dữ liệu sử dụng 4
2.2.1. Dữ liệu và thao tác chính 4
2.2.2. Quy ước và điều kiện hợp lệ 5
2.3. Nguyên lý thuật toán 6
2.3.1. Ý tưởng xử lý 6
2.3.2. Điều kiện bắt đầu và kết thúc 6
2.4. Cơ sở đánh giá thời gian và bộ nhớ 7
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 8
3.1. Phân tích yêu cầu chương trình 8
3.1.1. Hợp đồng input/output 8
3.1.2. Kiểm tra đầu vào 8
3.2. Thiết kế các hàm và dữ liệu 9
3.3. Thuật toán và mã nguồn chính 10
3.3.1. Giả mã theo cài đặt 10
3.3.2. Đoạn mã và giải thích 11
3.3.3. Xử lý tình huống đặc biệt 12
3.4. Lần vết ví dụ nhỏ 12
3.5. Giải thích tính đúng và điều kiện dừng 13
3.5.1. Điều kiện khởi đầu và duy trì 13
3.5.2. Kết thúc và output 14
3.6. Phân tích độ phức tạp 14
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 16
4.1. Môi trường và kế hoạch kiểm thử 16
4.1.1. Môi trường và phiên bản 16
4.1.2. Phương pháp xác định kỳ vọng 16
4.2. Bảng kết quả ít nhất 12 ca 17
4.3. Hai ví dụ trên phiếu 21
4.3.1. Ví dụ 1 — VD01 21
4.3.2. Ví dụ 2 — VD02 22
4.4. Hai ca sinh viên tự thiết kế 22

4.4.1. Ca TK01 22
4.4.2. Ca TK02 22
4.5. Ba ca biên phù hợp với đề tài 22
4.5.1. Ca BIEN01 22
4.5.2. Ca BIEN02 23
4.5.3. Ca BIEN03 23
4.6. Kiểm tra input không hợp lệ — LOI01 23
4.7. Kết quả toàn bộ kiểm thử 23
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 24
4.8.1. Dữ liệu và điều kiện khảo sát 24
4.8.2. Kết quả và giới hạn nhận xét 24
4.9. GitHub và chạy lại từ bản clone sạch 25
4.9.1. Phiên bản có thể đối chiếu 25
4.9.2. Clone sạch và tái hiện kết quả 26
4.9.3. Bằng chứng kho GitHub thật 27
4.10. Bộ ảnh minh chứng cá nhân 27
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 39
5.1. Mức hoàn thành so với phiếu 39
5.2. Giới hạn và bài học 40
5.3. Hướng phát triển phù hợp 40
TÀI LIỆU THAM KHẢO 41
PHỤ LỤC 42
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy 42
PHỤ LỤC B. Danh mục bằng chứng 43
PHỤ LỤC C. Tự kiểm và danh mục kiểm thử 44
PHỤ LỤC D. Hướng dẫn nộp Classroom 47

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Bảng P.3. Thuật ngữ và ký hiệu

Ký hiệu	**Ý nghĩa**
FIFO	First In, First Out: vào trước, ra trước.
BFS	Breadth-First Search: duyệt theo chiều rộng; với cây, xử lý theo mức.
ID	Định danh nút, thuộc 1..n; khác với giá trị lưu tại nút.
n	Số nút của cây, $0 \le n \le 100$.
L	Số mức. Cây không rỗng có L=D+1; cây rỗng có L=0.
D	Độ sâu lớn nhất tính bằng số cạnh từ gốc; gốc có độ sâu 0.
H	Chiều cao theo giáo trình tính bằng số nút trên đường dài nhất; H=D+1 khi không rỗng.
w_i	Số nút tại mức i, tức độ rộng mức i; tổng các w_i bằng n.
qmax	Số nút đang chờ lớn nhất trong FIFO chính; không phải số ô đã cấp phát.
C	Commit chứa mã/test/input dùng tạo kết quả và chụp ảnh.
S	Phiên bản mã và minh chứng được chốt để nộp; SHA ghi trong TXT nộp rời.
PASS / FAIL	Kết quả đối chiếu actual với expected; số phép kiểm khác số ca độc lập.

DANH SÁCH BẢNG

Bảng P.1. Phiếu chấm theo rubric chung iii
Bảng P.2. Vị trí đối chiếu sáu tiêu chí iii
Bảng P.3. Thuật ngữ và ký hiệu ix
Bảng 1.1. Mục tiêu của đề tài và sản phẩm kiểm được 1
Bảng 1.2. Hợp đồng input và output của bản cài đặt 2
Bảng 1.3. Hai ví dụ theo phiếu giao 3
Bảng 2.1. Cấu trúc dữ liệu và vai trò 5
Bảng 2.2. Các bất biến cấu trúc sử dụng 5
Bảng 3.1. Hợp đồng các hàm chính 9
Bảng 3.2. Các nhánh đặc biệt và hành vi thực tế 12
Bảng 3.3. Lần vết hàng đợi trên ví dụ 1 12
Bảng 3.4. Lần vết nhóm mức và kết quả 13
Bảng 3.5. Chi phí từng pha theo bản cài đặt 15
Bảng 4.1. Môi trường và quy trình chạy 16
Bảng 4.2. Đối chiếu năm ca bắt buộc của phiếu 17
Bảng 4.3. Thiết kế 13 ca đối chiếu đại diện 17
Bảng 4.4. Actual và vị trí minh chứng của 13 ca 20
Bảng 4.5. Tổng kết các bộ kiểm tra và cách tính 23
Bảng 4.6. Khảo sát Q1 Q2 Q3 trên hai dạng cây 24
Bảng 4.7. Kho mã và phiên bản đối chiếu 25
Bảng 5.1. Mức hoàn thành và bằng chứng 39
Bảng PL.1. Liên hệ ảnh với ca kiểm và kết quả 43
Bảng PL.2. Danh mục 38 nhóm kiểm thử thuật toán 44
Bảng PL.3. Danh mục 16 nhóm kiểm thử nhập liệu 45
Bảng PL.4. Tự kiểm sản phẩm trước khi nộp 46
Bảng PL.5. Bốn tệp theo quy định học phần 47

DANH SÁCH HÌNH

Hình 4.1. Ví dụ 1 trên phiếu 28
Hình 4.2. Ví dụ 2 trên phiếu 29
Hình 4.3. Ca tự thiết kế cây đầy bảy nút 30
Hình 4.4. Ca tự thiết kế hòa độ rộng 31
Hình 4.5. Ca biên cây rỗng 32
Hình 4.6. Ca biên dây phải bốn nút 33
Hình 4.7. Ca biên gốc có hai lá 34
Hình 4.8. Từ chối input không hợp lệ 35
Hình 4.9. Tổng kết bộ kiểm thử Julia 36
Hình 4.10. Kiểm phiên bản C và chạy lại tại thư mục clone 37
Hình 4.11. Repo và commit GitHub thật 38

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Duyệt cây theo mức là cách xử lý lần lượt các nút từ gốc xuống các mức sâu hơn. Khác với việc đi hết một nhánh trước, thứ tự này cho phép nhìn trực tiếp số nút ở từng mức và xác định mức tập trung nhiều nút nhất. Đề tài DT084 sử dụng một cây nhị phân có gốc chỉ định để rèn luyện biểu diễn liên kết, hàng đợi FIFO và quy tắc giữ thứ tự trái trước phải [1, 2].

Bài toán nhận danh sách nút thay vì một hình vẽ có sẵn. Chương trình cần kiểm tra danh sách có tạo thành đúng một cây hay không, sau đó in dãy duyệt, các nhóm cùng mức, độ rộng lớn nhất và mức đầu tiên đạt độ rộng đó. Vì ID và giá trị có thể khác nhau, cấu trúc cây phải được xác định bằng ID; không được giả định bản ghi thứ i luôn là nút i hoặc hai nút trùng giá trị là cùng một nút.

Yêu cầu chọn mức đầu tiên khi hòa và xử lý cây rỗng là hai phần nhỏ nhưng dễ gây sai lệch kết quả. Việc ghi rõ các trường hợp này ngay trong đặc tả giúp thiết kế kỳ vọng trước khi chạy và xây dựng các ca kiểm có mục đích.

1.2. Mục tiêu và sản phẩm

Bảng 1.1. Mục tiêu của đề tài và sản phẩm kiểm được

Mục tiêu	**Sản phẩm hoặc cách kiểm**
Biểu diễn và kiểm dữ liệu cây	Bảng (id, giá_trị, trái, phải), cấu trúc NutCay và validator từ chối dữ liệu sai.
Tự cài duyệt theo mức	Hai mảng queue và hai chỉ số; dãy ID/giá trị, các nhóm mức, trái trước phải.
Tính độ rộng và xử lý hòa	Dãy độ rộng; rộng nhất và mức nhỏ nhất đạt rộng nhất; quy ước rỗng rõ ràng.
Giải thích thuật toán	Giả mã, mã trọng tâm, truy vết ví dụ 1 và lập luận bất biến trong Chương 3.
Kiểm thử và khảo sát	Đủ ví dụ và ca phiếu, 13 ca đối chiếu đại diện, 139 phép kiểm Julia, khảo sát 3/7/31 nút.
Tái hiện và trình bày	README, menu/nhập bàn phím, GitHub, SHA C, 11 ảnh cá nhân và kết quả chạy lại từ clone.

Ngoài yêu cầu bắt buộc, chương trình đã cài phần mở rộng tính trung bình giá trị từng mức. Giao diện HTML mô phỏng các bước FIFO để hỗ trợ giải thích, còn kết quả thực thi và bộ kiểm thử chính sử dụng Julia. Phần bảo vệ trực tiếp chưa có kết quả chấm trong báo cáo này.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Theo phiếu, cây có $0 \le n \le 100$ nút; ID bao phủ 1..n; chỉ số con thuộc 0..n và 0 biểu diễn con rỗng. Có một gốc chỉ định và một cây liên thông; khi thêm con vào FIFO phải thêm trái trước phải. Dữ liệu có thể khai báo trực tiếp trong mã hoặc nhập từ bàn phím, không cần CSV, Excel hay tập dữ liệu ngoài [2].

Phiếu không ấn định kiểu cụ thể cho giá trị lưu ở nút. Bản cài đặt quy định giá trị là số thực hữu hạn biểu diễn được bằng Float64 hoặc chuỗi, không nhận Bool, số phức, NaN hay vô cực. Đây là hợp đồng giá trị của chương trình; ID/gốc/con luôn phải là số nguyên đúng miền. Nút khác ID vẫn có thể trùng giá trị.

Cây rỗng được biểu diễn bằng goc=0 và danh sách nút rỗng. Với cây không rỗng, gốc phải thuộc 1..n và không có cha. Bản ghi có thể xáo thứ tự; dựng cây theo ID làm kết quả không phụ thuộc thứ tự dòng nhập. Hàm đọc bàn phím dùng một token cho giá trị mỗi dòng, do đó chuỗi chứa khoảng trắng cần khai báo trực tiếp trong mã hoặc nhập qua quy tắc JSON của giao diện.

1.3.2. Kết quả và trường hợp đặc biệt

Bảng 1.2. Hợp đồng input và output của bản cài đặt

Thành phần	**Quy định**
goc, nut	goc là ID gốc; nut là vector/tuple các bộ bốn trường. Validator kiểm trước khi duyệt.
cac_muc	Dãy nhóm giá trị từ mức 0; tổng số giá trị trong mọi nhóm bằng n.
day_theo_muc, thu_tu_id	Dãy giá trị và dãy ID theo thứ tự FIFO; mỗi ID xuất hiện đúng một lần.

Thành phần	Quy định
do_rong	Số nút của từng mức; không tính khoảng cách ô trống trong cách vẽ cây.
rong_nhat, muc	Rộng nhất và mức nhỏ nhất đạt độ rộng đó. Rỗng trả 0 và nothing.
Bộ đếm queue	Số lượt vào/ra FIFO chính và số nút đang chờ lớn nhất; không cộng queue của validator.
trung_binh	Mức toàn số trả Float64 gần đúng; mức chứa chuỗi trả nothing; cây rỗng trả dãy rỗng.
Dữ liệu không hợp lệ	Ném ArgumentError kèm lý do. Menu cho chọn lại; lệnh demo thông thường trả mã lỗi khác 0.

Bảng 1.3. Hai ví dụ theo phiếu giao

Ví dụ và input	Kỳ vọng tính từ cây
Ví dụ 1: goc=1; nut=[(1,10,2,3),(2,20,4,0),(3,30,0,0),(4,40,0,0)].	cac_muc=[[10],[20,30],[40]]; dãy [10,20,30,40]; độ rộng [1,2,1]; rộng nhất 2, mức 1.
Ví dụ 2: goc=1; nut=[(1,8,0,0)].	cac_muc=[[8]]; dãy [8]; độ rộng [1]; rộng nhất 1, mức 0.

Nếu nhiều mức cùng rộng nhất, kết quả phải giữ mức nhỏ nhất. Cây dây bốn nút vì thế trả mức 0, không phải mức cuối. Dữ liệu sai như gốc có cha, con chung hoặc phần rời phải bị từ chối; không coi một danh sách có chu trình là cây hợp lệ rồi để FIFO chạy vô hạn.

1.4. Cách thực hiện và cấu trúc báo cáo

Quy trình đi từ phiếu và giáo trình tới đặc tả, cài đặt, kỳ vọng độc lập, kiểm thử và minh chứng theo commit. Chương 2 trình bày lý thuyết; Chương 3 giải thích thiết kế, truy vết, tính đúng và chi phí; Chương 4 đối chiếu kết quả, khảo sát và GitHub; Chương 5 nêu mức hoàn thành và giới hạn. Phần hỗ trợ của ChatGPT/Codex được khai báo trong Lời cam đoan.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Cây là mô hình phân cấp có một gốc và quan hệ cha–con. Trong cây có gốc, mỗi nút khác gốc có đúng một cha và có đúng một đường đi từ gốc tới nút đó. Cây nhị phân giới hạn mỗi nút có tối đa hai con phân biệt theo vị trí trái và phải. Một nút không có con là nút lá; danh sách trỏ đến cùng một nút con từ hai cha không còn mô tả cây theo hợp đồng đề tài [1, trang in 161–163].

Mức của một nút là số cạnh trên đường từ gốc tới nút: gốc ở mức 0, con của nút mức i ở mức i+1. Ký hiệu L là số mức và D là độ sâu lớn nhất tính bằng số cạnh, khi không rỗng L=D+1. Giáo trình định nghĩa chiều cao theo số nút trên đường dài nhất nên H=D+1; báo cáo dùng tên "độ sâu lớn nhất" cho cột số cạnh của khảo sát để tránh nhầm hai quy ước [1, trang in 161].

Độ rộng w_i của mức i là số nút thực sự có ở mức đó. Các mức phân hoạch tập nút nên tổng w_i bằng n. Độ rộng lớn nhất là $\max(w_i)$, và mức cần trả là chỉ số i nhỏ nhất đạt giá trị này. Cây rỗng không có mức; chọn rộng nhất bằng 0 và mức không tồn tại là quy ước biên của bài toán.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Hàng đợi là cấu trúc vào trước ra trước: phần tử thêm sớm nhất trong số phần tử còn chờ được lấy trước. Hai thao tác chính là đưa vào cuối và lấy ở đầu. Với duyệt mức, một mục queue cần mang cả ID nút lẫn mức; hai thông tin này có thể lưu trong cùng một cặp hoặc trong hai mảng song song [1, mục 5.4, trang in 130–132].

Bản Julia chọn hai mảng hang_id, hang_muc, mỗi mảng n ô. dau chỉ ô cần lấy kế tiếp, cuoi chỉ ô đã ghi cuối cùng. Sau khi lấy chỉ tăng dau, không xóa phần tử đầu vector và không dịch các ô còn lại. Vì mỗi nút chỉ thêm một lần nên cuoi không vượt n; đây là FIFO tuyến tính cấp phát theo số nút, không phải hàng đợi vòng sức chứa tái sử dụng.

Bảng 2.1. Cấu trúc dữ liệu và vai trò

Cấu trúc	Biểu diễn và thao tác
NutCay và mảng cay	Mỗi nút có ID, giá trị, ID con trái và ID con phải; 0 biểu diễn con rỗng. Mảng truy cập trực tiếp bằng ID giữ nguyên quan hệ cây dù thứ tự bản ghi đầu vào bị xáo trộn. Giá trị trùng không làm hai nút trở thành một.
Các mảng validator	co_id đánh dấu ID đã gặp; bac_vao đếm liên kết cha; con_trai và con_phai lưu cấu trúc theo ID. da_tham và hang_kt phục vụ duyệt kiểm liên thông. Các mảng có chiều dài n nên việc xác thực có thời gian và bộ nhớ O(n), không phụ thuộc giả thiết chi phí bảng băm.
FIFO hai mảng	hang_id và hang_muc lần lượt lưu ID và mức tại cùng vị trí. dau là vị trí lần lấy tiếp theo, cuoi là vị trí đã ghi cuối cùng; đoạn dau..cuoi chứa các nút đang chờ. Thêm tăng cuoi, lấy tăng dau, mỗi thao tác O(1) và không dịch các phần tử còn lại.
Kết quả và bộ đếm	cac_muc lưu các nhóm theo mức; day_theo_muc và thu_tu_id lưu thứ tự đã xử lý. so_nut_vao_queue, so_nut_ra_queue và max_queue_len hỗ trợ kiểm bất biến và khảo sát thao tác. max_queue_len là số nút đang chờ lớn nhất, không phải số ô bộ nhớ đã cấp phát.

2.2.2. Quy ước và điều kiện hợp lệ

Bảng nút được đánh địa chỉ theo ID, còn giá trị chỉ là dữ liệu mang theo. Trước khi truy cập cay[id], validator bảo đảm ID đủ 1..n và không trùng. Chỉ số con được kiểm miền trước khi ép sang Int; đây là lý do một BigInt chỉ số quá lớn nhận ArgumentError theo hợp đồng thay vì lỗi ép kiểu ngoài dự kiến.

Đếm bậc vào giúp phát hiện con có nhiều liên kết cha và gốc có cha. Sau kiểm tra cục bộ, một lượt duyệt đánh dấu từ gốc phải tới đủ n nút. Điều kiện toàn cục này loại bỏ nút rời hoặc một chu trình nằm ở thành phần không nối với gốc mà chỉ kiểm từng bản ghi chưa phát hiện được.

Bảng 2.2. Các bất biến cấu trúc sử dụng

Bất biến	Ý nghĩa khi cài đặt
ID duy nhất và đủ 1..n	Mỗi ô cay[id] được khởi tạo; không dựa vào thứ tự dòng.

Bất biến	Ý nghĩa khi cài đặt
Một gốc, mỗi nút khác gốc một cha	Không thêm cùng một nút từ hai cha; gốc có bậc vào 0.
Tất cả nút tới được từ gốc	Không bỏ quên thành phần rời; tổng số nút được duyệt bằng n.
Queue chứa nút đã phát hiện, chưa xử lý	Đoạn dau..cuoi là nội dung FIFO logic; các ô trước dau là dữ liệu đã xử lý.
Mức queue không giảm	Nút cha được xử lý trước mức con; nhóm kết quả đúng độ sâu.

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Mục 7.3.2 của giáo trình trình bày duyệt theo mức bằng hàng đợi: đưa gốc vào hàng, lặp việc lấy nút đầu hàng, xử lý nút và thêm các con vào cuối [1, trang in 165; trang PDF 177]. Ví dụ trong giáo trình dùng deque của Python; chương trình đề tài chuyển ý tưởng sang Julia và tự cài FIFO mảng, không gọi sẵn một hàm duyệt cây để thay phần phải thực hiện.

Khi lấy mục (u,m), chương trình đưa giá trị u vào nhóm m, rồi thêm con trái và con phải tồn tại với mức m+1. Các cha cùng mức đã được lấy theo FIFO, nên thứ tự con của họ được giữ. Khi queue rỗng, độ dài từng nhóm là độ rộng; tìm vị trí đầu tiên đạt maximum giải quyết quy tắc hòa.

2.3.2. Điều kiện bắt đầu và kết thúc

Cây rỗng được trả kết quả ngay trước khi thêm gốc. Với cây hợp lệ không rỗng, khởi tạo dau=cuoi=1, hang_id[1]=goc, hang_muc[1]=0. Vòng lặp còn làm việc khi dau≤cuoi; dau>cuoi nghĩa là không còn nút đang chờ.

Mỗi bước lấy làm dau tăng một. Vì một cây có hữu hạn n nút, mỗi nút khác gốc có duy nhất một cha, tổng số lần thêm và lấy đều bằng n. Điều kiện dừng gắn trực tiếp với việc xử lý đủ nút, không phụ thuộc giá trị số hay chuỗi lưu trong nút.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Phân tích dùng n là kích thước cấu trúc, L là số mức và chi phí truy cập mảng/so sánh chỉ số xem là hằng số. push! vào vector được tính theo chi phí khấu hao. Nếu tính chuỗi rất dài hoặc số nhiều chữ số, cần cộng thêm chi phí đọc, chuyển đổi và hiển thị giá trị; kết luận O(n) không có nghĩa các chuỗi tùy ý đều được xử lý trong một đơn vị thời gian.

Bộ đếm thực nghiệm đo lượt vào và ra của FIFO chính cùng số nút đang chờ lớn nhất. Chúng không đếm queue kiểm liên thông của validator. Khảo sát này cho thấy đặc điểm số thao tác của lõi duyệt; chi phí toàn chương trình được kết luận bằng cách cộng các pha phân tích trong Mục 3.6, không lấy cột Time của bộ test làm số đo benchmark.

Ở một thời điểm, queue có thể đồng thời chứa phần còn lại của mức i và các con ở mức i+1. Vì vậy không đồng nhất qmax với độ rộng lớn nhất cho mọi cây. Riêng các cây đầy 3,7,31 nút được chọn trong khảo sát, mức cuối chi phối qmax; cây dây chỉ chờ một nút nhưng mảng vẫn cấp phát n ô.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

Chương trình tách kiểm tra dữ liệu, dựng bảng nút theo ID, duyệt FIFO và tổng hợp thống kê. Phần bắt buộc được giải bằng một thuật toán duyệt theo mức; hàm trung bình là mở rộng tùy chọn của cùng kết quả, không phải thuật toán thứ hai để thay đổi bài toán. Cấu trúc hàng đợi do chương trình tự cài bằng mảng và hai chỉ số.

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Các quy ước ở Bảng 1.2 được thực hiện thống nhất trong hàm giai_bai và các chế độ demo/nhập liệu. Mức dùng chỉ số từ 0 theo phiếu; chỉ số vector Julia bắt đầu từ 1, nên nhóm của mức m được truy cập tại cac_muc[m+1]. ID được giữ riêng trong thu_tu_id để phân biệt các nút trùng giá trị.

Tầng đọc bàn phím kiểm dòng đầu ngay lập tức: n âm, n>100 hoặc gốc không phù hợp đều bị từ chối trước khi yêu cầu nhập n dòng. Sau đó mỗi dòng phải có đúng bốn trường; thiếu dòng, ID/con không nguyên hoặc số không hữu hạn được báo lỗi. Dữ liệu cấu trúc hoàn chỉnh tiếp tục được kiểm bằng cùng validator của API.

3.1.2. Kiểm tra đầu vào

Kiểm $0 \le n \le 100$; cây rỗng chỉ nhận goc=0. Với cây có nút, kiểm bộ bốn trường, ID đủ 1..n, con trong 0..n, giá trị hợp lệ, không khuyên hoặc hai con trùng, mỗi con tối đa một liên kết cha và gốc không có cha. Sau đó duyệt từ gốc, đánh dấu ngay khi đưa vào hàng; yêu cầu tất cả nút đã được tới. Miền số được kiểm trước Int() để tránh lỗi ép kiểu trên BigInt quá lớn.

Việc kết hợp kiểm bậc vào và kiểm tới được là cần thiết. Một cây rời với hai nút lá riêng có thể thỏa từng chỉ số con nhưng không có một gốc bao phủ toàn bộ; một chu trình rời cũng không nhất thiết làm gốc có cha. Validator chỉ chấp nhận sau khi bước kiểm liên thông đánh dấu đủ n ID.

```
KIEM_TRA(goc, nut)
  Kiểm kiểu danh sách, kiểu gốc và 0 ≤ n ≤ 100.
  Nếu n = 0: yêu cầu goc = 0; trả hợp lệ.
  Yêu cầu goc thuộc 1..n.
  Tạo các mảng co_id, bac_vao, con_trai, con_phai theo ID.
  Với mỗi bản ghi (id, giá_trị, trái, phải):
    Kiểm đủ 4 trường, ID/con nguyên, miền chỉ số và giá trị.
    Từ chối ID lặp, khuyên, hai con trùng hoặc con có hơn một cha.
    Lưu hai con theo ID và tăng bậc vào của con tồn tại.
```

```
Yêu cầu đủ các ID 1..n và bậc vào của gốc bằng 0.
Duyệt từ gốc bằng hàng đợi, đánh dấu ngay lúc thêm.
Từ chối nếu gặp nút đã đánh dấu hoặc còn ID chưa tới được.
Trả hợp lệ.
```

3.2. Thiết kế các hàm và dữ liệu

Bảng 3.1. Hợp đồng các hàm chính

Hàm	**Tiền điều kiện**	**Hậu điều kiện**
kiem_tra_dau_vao(goc, nut)	Hàm nhận dữ liệu chưa được tin cậy. nut phải là vector hoặc tuple; goc phải là số nguyên và không phải Bool.	Trả true nếu dữ liệu mô tả đúng một cây nhị phân theo hợp đồng; nếu sai thì ném ArgumentError trước khi thuật toán duyệt chính bắt đầu.
khoi_tao_cay(nut)	Bảng nút đã qua kiem_tra_dau_vao với gốc tương ứng. Không gọi hàm dựng cây trực tiếp trên dữ liệu chưa kiểm tra.	Trả mảng cay có n phần tử NutCay, cay[id] chứa đúng bản ghi của ID đó; thứ tự các dòng trong nut không ảnh hưởng kết quả. Cây rỗng trả NutCay[].
duyet_muc(goc, nut)	Nhận dữ liệu công khai và tự gọi validator; đầu vào hợp lệ phải tuân theo hợp đồng cây.	Trả cac_muc, day_theo_muc, thu_tu_id và ba bộ đếm. Mỗi ID xuất hiện đúng một lần, nút được nhóm theo độ sâu tính từ gốc ở mức 0; trong mỗi mức, con trái đứng trước con phải theo thứ tự các cha.
do_rong(cac_muc)	cac_muc là danh sách các nhóm nút đã được duyệt đúng mức.	Trả w với w[i+1] là số nút ở mức i. Với cây rỗng, w là dãy rỗng.
muc_rong_nhat(cac_muc)	Các nhóm đã có thứ tự mức tăng dần từ 0.	Trả (rong_nhat, muc); nếu rỗng thì (0,nothing). Khi hòa, muc là mức nhỏ nhất đạt độ rộng lớn nhất.
trung_binh_tung_muc(cac_muc)	Giá trị của mỗi nhóm là số thực hữu hạn biểu diễn được bằng Float64 hoặc chuỗi, không nhận Bool/số phức/NaN/Inf.	Mỗi mức toàn số có kết quả Float64; mức chứa chuỗi hoặc nhóm rỗng trả nothing. Cây rỗng trả Float64[].

Hàm	Tiền điều kiện	Hậu điều kiện
giai_bai(goc, nut)	Nhận input công khai; việc từ chối dữ liệu sai được thực hiện qua duyet_muc.	Trả named tuple kết hợp input, n, các nhóm, dãy duyệt, độ rộng, mức rộng nhất, bộ đếm queue và trung bình. Trước khi trả, kiểm tổng độ rộng bằng n, dãy duyệt đủ n phần tử và số lượt vào/ra queue bằng n khi n>0.
doc_cay(io=stdin; out=stdout)	Luồng nhập chứa dòng đầu n goc và đủ n dòng id gia_tri trai phai. Giá trị nhập mỗi dòng là một token, chuỗi không có khoảng trắng.	Trả (goc,nut) đã được validator chấp nhận hoặc ném ArgumentError. Kiểm n và gốc ngay sau dòng đầu, trước khi yêu cầu nhập các nút.

Các hàm tách theo trách nhiệm để có thể kiểm tra riêng. khoi_tao_cay là hàm hỗ trợ có tiền điều kiện dữ liệu đã xác thực; duyet_muc và giai_bai là điểm vào công khai có gọi validator. Việc sử dụng Vector, struct, length, push!, maximum và findfirst của Julia phục vụ lưu trữ/tổng hợp, không thay thế vòng lặp FIFO tự cài.

NutCay giữ ID, giá trị và hai ID con. Các mảng kiểm tra có chiều dài n, nhờ đó truy cập bậc vào và con theo ID có thời gian xác định O(1). Kết quả dùng named tuple có tên trường rõ ràng để demo, test và khảo sát cùng đọc một giao diện; không lưu queue toàn cục giữa các lần gọi.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

```
DUYET_MUC(goc, nut)
  KIEM_TRA(goc, nut).
  Nếu cây rỗng: trả các dãy rỗng và bộ đếm 0.
  Dựng cay[id]; tạo hang_id[1..n], hang_muc[1..n].
  dau ← 1; cuoi ← 1; hang_id[1] ← goc; hang_muc[1] ← 0.
  Khởi tạo các nhóm, dãy giá trị và dãy ID rỗng.
  vao ← 1; ra ← 0; queue_max ← 1.
  Trong khi dau ≤ cuoi:
    queue_max ← max(queue_max, cuoi – dau + 1).
    u ← hang_id[dau]; m ← hang_muc[dau]; dau ← dau + 1; ra ← ra + 1.
    Ghi giá trị u và ID u vào hai dãy duyệt.
    Tạo nhóm m nếu chưa có, rồi thêm giá trị u vào nhóm m.
    Với v lần lượt là con trái rồi con phải của u:
      Nếu v ≠ 0: cuoi ← cuoi + 1;
        hang_id[cuoi] ← v; hang_muc[cuoi] ← m + 1; vao ← vao + 1.
  Trả các nhóm, các dãy và bộ đếm.
```

```
CHON_MUC_RONG_NHAT(cac_muc)
  Nếu không có nhóm: trả (0, không tồn tại).
  w ← độ dài từng nhóm theo thứ tự mức.
  rong_nhat ← giá trị lớn nhất của w.
  i ← chỉ số đầu tiên thỏa w[i] = rong_nhat.
  Trả (rong_nhat, i – 1).
```

3.3.2. Đoạn mã và giải thích

Đoạn sau trích lõi vòng lặp trong src/DeTai.jl tại commit C [3]. Hai mảng chứa ID và mức cùng chỉ số; bộ đếm max_queue_len được cập nhật ở đầu mỗi vòng khi queue đang chờ đã phản ánh các con vừa thêm ở vòng trước.

```
    while dau <= cuoi
        do_dai_hien_tai = cuoi - dau + 1
        if do_dai_hien_tai > max_queue_len
            max_queue_len = do_dai_hien_tai
        end
        u_id = hang_id[dau]
        muc_u = hang_muc[dau]
        dau += 1
        so_nut_ra_queue += 1
        nut_u = cay[u_id]
        push!(day_theo_muc, nut_u.gia_tri)
        push!(thu_tu_id, u_id)
        while length(cac_muc) <= muc_u
            push!(cac_muc, Any[])
        end
        push!(cac_muc[muc_u + 1], nut_u.gia_tri)
        if nut_u.trai != 0
            cuoi += 1
            hang_id[cuoi] = nut_u.trai
            hang_muc[cuoi] = muc_u + 1
            so_nut_vao_queue += 1
        end
        if nut_u.phai != 0
            cuoi += 1
            hang_id[cuoi] = nut_u.phai
            hang_muc[cuoi] = muc_u + 1
            so_nut_vao_queue += 1
        end
    end
```

dau tăng sau mỗi lần lấy nên thao tác không dịch phần tử. cuoi chỉ tăng khi gặp con tồn tại, và validator bảo đảm mỗi nút khác gốc chỉ có một cha. Vòng while mở rộng cac_muc vừa đủ tới mức đang xử lý; mỗi mức được tạo một lần nên tổng số nhóm L không làm chi phí tăng thành O(nL).

Khi xử lý hòa, muc_rong_nhat tính ds_rong, lấy max_w=maximum(ds_rong), rồi idx=findfirst(==(max_w),ds_rong) và trả idx-1. Chuyển đổi này giữ mức 0 ở gốc và tránh chọn nhầm mức cuối cùng khi nhiều mức cùng lớn nhất.

Permalink mã thuật toán: src/DeTai.jl tại C

3.3.3. Xử lý tình huống đặc biệt

Bảng 3.2. Các nhánh đặc biệt và hành vi thực tế

Tình huống	**Cách xử lý**
Cây rỗng	Trả các dãy rỗng và bộ đếm 0; rộng nhất 0, mức nothing.
Một nút	Gốc ở nhóm 0; thêm/lấy một lần; rộng nhất 1 tại mức 0.
Hòa độ rộng	Dùng findfirst, giữ mức nhỏ nhất đạt maximum.
Bản ghi xáo thứ tự hoặc giá trị trùng	Dựng cay[id], duyệt bằng ID, không dùng vị trí dòng hoặc giá trị làm liên kết.
Sai ID/con, gốc có cha, con chung, chu trình/rời	ArgumentError từ validator trước khi tạo kết quả chính.
Mức chứa chuỗi	Duyệt và nhóm bình thường; trung bình trả nothing.
Trung bình số lớn	Chuyển Float64, chia trước khi cộng và chuẩn hóa dự phòng; kết quả vẫn là số gần đúng.
Demo A08	Bắt đúng ArgumentError dự kiến và trả thành công vì đã từ chối dữ liệu sai; lỗi khác được ném lại.

3.4. Lần vết ví dụ nhỏ

Chọn đúng ví dụ 1: goc=1; nut=[(1,10,2,3),(2,20,4,0),(3,30,0,0),(4,40,0,0)]. Kỳ vọng được lập từ quan hệ gốc 10, hai con 20/30 và cháu 40 của nút 20. Một mục (id,m) biểu diễn ID và mức. Bảng 3.3 ghi trạng thái sau khi lấy nút và thêm các con; bước 0 là khởi tạo.

Bảng 3.3. Lần vết hàng đợi trên ví dụ 1

Bước	**Nút lấy**	**Con thêm**	**dau/cuoi**	**Queue đang chờ**
0 — khởi tạo	—	gốc (1,0)	1/1	[(1,0)]
1	ID 1, giá trị 10	(2,1), (3,1)	2/3	[(2,1),(3,1)]
2	ID 2, giá trị 20	(4,2)	3/4	[(3,1),(4,2)]
3	ID 3, giá trị 30	không	4/4	[(4,2)]
4	ID 4, giá trị 40	không	5/4	[]

Bảng 3.4. Lần vết nhóm mức và kết quả

Bước	Dãy giá trị đã xử lý	Các nhóm đã tích lũy	Độ rộng tạm thời
0 — khởi tạo	[]	[]	[]
1	[10]	[[10]]	[1]
2	[10,20]	[[10],[20]]	[1,1]
3	[10,20,30]	[[10],[20,30]]	[1,2]
4	[10,20,30,40]	[[10],[20,30],[40]]	[1,2,1]

Kết thúc sau 4 lần lấy; enqueue=dequeue=4; số nút đang chờ lớn nhất bằng 2. Các nhóm [[10],[20,30],[40]] cho độ rộng [1,2,1], nên rộng nhất 2 tại mức 1. Hai nút 20 và 30 cùng mức được xử lý trước 40 ở mức tiếp theo.

Sau bước 2, nhóm mức 1 mới chứa 20 nên độ rộng tạm là [1,1]; đây chưa phải kết quả cuối. Sau bước 3, nút 30 hoàn tất nhóm mức 1, và mức tiếp theo mới được xử lý. Phân biệt trạng thái trung gian với kết quả cuối giúp phát hiện lỗi ghi nhóm quá sớm hoặc đưa con vào sai thứ tự.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Dữ liệu hợp lệ trước khi duyệt chính. Validator kiểm ID đủ và duy nhất trong 1..n nên mỗi ô cay[id] đều được khởi tạo. Chỉ số con đã nằm trong 0..n, do đó mọi con khác 0 đều truy cập đúng miền. Ràng buộc mỗi con có tối đa một cha và gốc không có cha loại bỏ dùng chung nút và liên kết quay vào gốc. Bước duyệt đánh dấu kết hợp yêu cầu tới đủ n nút loại bỏ chu trình hoặc thành phần rời còn lại. Vì vậy thuật toán chính làm việc trên một cây nhị phân liên thông, không cần dùng giá trị nút để phát hiện trùng.

Bất biến thứ tự hàng đợi. Trước mỗi lần lặp, đoạn hang_id[dau..cuoi] chứa đúng các nút đã phát hiện nhưng chưa xử lý, theo thứ tự mức không giảm; hang_muc lưu đúng độ sâu của từng nút. Các nút cùng mức được giữ theo thứ tự trái sang phải do các cha được lấy FIFO và từng cha thêm con trái trước con phải. Những nút trước dau đã được ghi đúng một lần vào dãy duyệt và đúng nhóm mức.

Khởi tạo và bảo toàn. Ban đầu queue chỉ chứa gốc ở mức 0 nên bất biến đúng. Khi lấy nút u ở đầu hàng có mức m, việc ghi u vào nhóm m giữ đúng quan hệ mức. Hai con của u, nếu có, thực sự ở mức m+1; chúng được thêm cuối hàng theo thứ tự trái trước phải nên không vượt các nút đang chờ ở mức m hay các nút cùng mức m+1 đã được phát hiện trước. Mỗi nút khác gốc có đúng một cha trong cây nên chỉ được thêm khi cha đó được xử lý.

3.5.2. Kết thúc và output

Kết thúc và đủ nút. Chỉ số dau tăng một sau mỗi lần lấy. Có n nút và mỗi nút được đưa vào hàng tối đa một lần; mọi nút tới được từ gốc nên cuối cùng được đưa vào và lấy ra đúng một lần. Vòng lặp kết thúc sau n lượt lấy, với dau>cuoi, hai bộ đếm đều bằng n và tổng kích thước các nhóm bằng n. Cây rỗng được xử lý trước lúc tạo gốc giả nên trả dãy rỗng, độ rộng 0 và không có mức rộng nhất.

Độ rộng lớn nhất và xử lý hòa. Các nhóm được lưu theo thứ tự mức từ 0 nên độ dài nhóm i+1 chính là số nút của mức i. maximum trả đúng độ rộng lớn nhất. findfirst tìm chỉ số đầu tiên của độ rộng đó; trừ một đổi sang số mức từ 0. Do đó khi nhiều mức cùng lớn nhất, hàm luôn giữ mức nhỏ nhất theo yêu cầu.

Giới hạn của kiểm thử. Các assertion kiểm tra những ví dụ và ca hồi quy cụ thể, còn chứng minh trên dùng bất biến cho mọi cây trong miền hợp đồng. 121 phép kiểm thuật toán và 18 phép kiểm nhập liệu đã đạt là bằng chứng thực nghiệm, không được diễn giải thành kiểm vét cạn mọi cây hay bằng chứng thay thế lập luận tính đúng.

Trung bình theo mức là phép tổng hợp sau khi nhóm đúng: với mức có k giá trị số, chương trình cộng các giá trị đã chia k. Việc chọn Float64 và nhánh chuẩn hóa nhằm tránh tràn trung gian trong phạm vi hợp đồng, không chứng minh kết quả số thực chính xác tuyệt đối. Giới hạn làm tròn được giữ riêng với tính đúng của thứ tự duyệt và độ rộng nguyên.

3.6. Phân tích độ phức tạp

Trong các bảng chi phí, n là số nút và L là số mức. Với cây không rỗng, L=D+1 và L≤n; cây rỗng được trả sớm. Thao tác mảng và thao tác giá trị có kích thước cố định xem là O(1); push! được tính theo chi phí khấu hao.

Bảng 3.5. Chi phí từng pha theo bản cài đặt

Pha	**Thời gian**	**Bộ nhớ phụ**	**Cơ sở**
Kiểm bảng và liên thông	O(n)	O(n)	Quét n bản ghi, tối đa hai con mỗi nút; các mảng theo ID và một lượt kiểm liên thông.
Dựng cây theo ID	O(n)	O(n)	Đặt mỗi bản ghi vào cay[id] một lần; có n cấu trúc NutCay.
FIFO và tạo kết quả mức	O(n)	O(n)	Mỗi nút vào/ra một lần; hai mảng queue n ô, các dãy ID/giá trị và tổng nhóm n phần tử. push! tính khấu hao.
Độ rộng và chọn mức	O(L)	O(L)	Có L nhóm, length của vector O(1); dãy độ rộng được quét một số lần hằng số.
Trung bình các mức	O(n)	O(L)	Mỗi giá trị được quét một số lần hằng số; dãy trung bình có L phần tử.
Toàn bộ giai_bai	O(n)	O(n)	Cộng các pha vẫn tuyến tính; giả sử chi phí giá trị/chuyển đổi cố định như đã nêu.

Các bộ đếm thực nghiệm chỉ đo FIFO chính; queue kiểm liên thông của validator không được cộng vào chúng, nhưng được tính trong phân tích tổng O(n). Cây dây có qmax=1 vẫn dùng hai mảng queue n ô, vì vậy bộ nhớ bản cài không trở thành O(1). Kết luận O(n) tính cả validator, bảng cây, queue và các dãy kết quả; mỗi thao tác O(1) lặp n lần cho thời gian tuyến tính.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Bảng 4.1. Môi trường và quy trình chạy

Thành phần	**Thông tin thực tế**
Hệ điều hành và shell	Windows; Windows PowerShell, thể hiện trên ảnh cá nhân.
Runtime chính	Julia 1.12.6; chạy --startup-file=no để không phụ thuộc thiết lập khởi động cá nhân.
Phụ thuộc Julia	Thư viện chuẩn Test và Printf; không cài gói ngoài, không cần tập dữ liệu ngoài.
Kiểm tra HTML	Node với mô hình DOM trong test/gui_tests.cjs: 24/24 ca logic đạt; chưa dùng kết quả này để kết luận bố cục trên mọi trình duyệt.
Bản mã chạy/chụp C	ccaf13ab5c6bca2a9affc669f42508e7d0f6848a
Input và expected	Khai báo trong demo.jl, test/runtests.jl, test/input_tests.jl; các ca bàn phím được kiểm bằng IOBuffer.
Bằng chứng	A01–A11; docs/logs/clone_C.log, clone_C_gui.log, kiem_tra_toan_bo.log; các nhật ký kiểm S được lưu rời sau khi chốt ảnh.
Ngày tổng hợp báo cáo	28/10/2026. Hạn nộp và lịch bảo vệ theo thông báo Classroom, không suy ra từ ngày sửa tệp.

4.1.2. Phương pháp xác định kỳ vọng

Kỳ vọng các cây nhỏ được xác định bằng sơ đồ quan hệ cha–con: liệt kê gốc, sau đó các con từ trái sang phải theo từng mức, rồi đếm số nút. Các mức hòa được xét theo chỉ số nhỏ nhất. Đối với dữ liệu sai, expected là ArgumentError vì vi phạm hợp đồng đã nêu, không phải một dãy kết quả duyệt.

Tệp test ghi các expected trực tiếp bằng giá trị hoặc công thức cấu tạo cây, không gọi lại giai_bai để tạo đáp án cho chính nó. Các assertion về tổng độ rộng và lượt queue

kiểm thêm bất biến; chúng không thay đổi chiếu giá trị cụ thể. Bảng catalog in lại input, expected, actual và PASS của 12 ca đã có, nên không cộng thành 12 ca mới [3].

4.2. Bảng kết quả ít nhất 12 ca

Bảng 4.2. Đối chiếu năm ca bắt buộc của phiếu

Ca trên phiếu	**Ca trong chương trình**	**Bằng chứng**
1. Cây rỗng	T03 / demo A05	Hình 4.5; rộng 0, không có mức.
2. Một nút	T02 / demo A02	Hình 4.2; rộng 1, mức 0. Ca này đồng thời là ví dụ 2, không đếm hai lần.
3. Gốc có hai lá	T04 / demo A07	Hình 4.7; rộng 2, mức 1.
4. Dây phải bốn nút	T05 / demo A06	Hình 4.6; độ rộng [1,1,1,1], chọn 0.
5. Cây đầy bảy nút	T06 / demo A03	Hình 4.3; độ rộng [1,2,4].

Bảng 4.3. Thiết kế 13 ca đối chiếu đại diện

ID và đối tượng	**Input đủ tái hiện**	**Expected độc lập**
T01 Ví dụ 1 trên phiếu giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,10,2,3),(2,20,4,0),(3,30,0,0),(4,40,0,0)]	cac_muc=[[10],[20,30],[40]]; dãy=[10,20,30,40]; độ rộng=[1,2,1]; rộng nhất=2; mức=1; vào=ra=4; queue_max=2.
T02 Ví dụ 2: một nút giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,8,0,0)]	cac_muc=[[8]]; độ rộng=[1]; rộng nhất=1; mức=0; vào=ra=1; queue_max=1.

ID và đối tượng	Input đủ tái hiện	Expected độc lập
T03 Cây rỗng giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=0; nut=[]	Các dãy rỗng; rộng nhất=0; mức=nothing; vào=ra=0; queue_max=0.
T04 Gốc có hai lá giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,100,2,3),(2,200,0,0),(3,300,0,0)]	cac_muc=[[100],[200,300]]; độ rộng=[1,2]; rộng nhất=2; mức=1; queue_max=2.
T05 Dây phải bốn nút giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,10,0,2),(2,20,0,3),(3,30,0,4),(4,40,0,0)]	cac_muc=[[10],[20],[30],[40]]; độ rộng=[1,1,1,1]; rộng nhất=1; mức=0; queue_max=1.
T06 Cây đầy bảy nút giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,1,2,3),(2,2,4,5),(3,3,6,7),(4,4,0,0),(5,5,0,0),(6,6,0,0),(7,7,0,0)]	cac_muc=[[1],[2,3],[4,5,6,7]]; độ rộng=[1,2,4]; rộng nhất=4; mức=2; queue_max=4.
D04 Hai mức hòa độ rộng (TK02, A04) giai_bai qua demo	goc=1; nut=[(1,10,2,3),(2,20,4,0),(3,30,0,5),(4,40,0,0),(5,50,0,0)]	cac_muc=[[10],[20,30],[40,50]]; độ rộng=[1,2,2]; rộng nhất=2; mức=1.
T07 Dây trái với giá trị chuỗi giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,"A",2,0),(2,"B",3,0),(3,"C",4,0),(4,"D",0,0)]	cac_muc=[["A"],["B"],["C"],["D"]]; dãy=["A","B","C","D"]; độ rộng=[1,1,1,1]; rộng nhất=1; mức=0; queue_max=1.

ID và đối tượng	Input đủ tái hiện	Expected độc lập
T08 Dây trái/phải xen kẽ giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,10,2,0),(2,20,0,3),(3,30,4,0),(4,40,0,0)]	cac_muc=[[10],[20],[30],[40]]; ID=[1,2,3,4]; độ rộng=[1,1,1,1]; rộng nhất=1; mức=0; queue_max=1.
T09 Ba mức hòa độ rộng giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; nut=[(1,1,2,3),(2,2,4,0),(3,3,0,5),(4,4,6,0),(5,5,0,7),(6,6,0,0),(7,7,0,0)]	cac_muc=[[1],[2,3],[4,5],[6,7]]; độ rộng=[1,2,2,2]; rộng nhất=2; mức=1.
T10 Cây đầy mười lăm nút giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=1; với i=1..15: nut[i]=(i,i, 2i nếu 2i≤15 ngược lại 0, 2i+1 nếu 2i+1≤15 ngược lại 0).	Độ rộng=[1,2,4,8]; rộng nhất=8; mức=3; queue_max=8; vào=ra=15.
T13 Gốc khác 1, bản ghi đảo thứ tự và giá trị lặp giai_bai / duyet_muc / do_rong / muc_rong_nhat	goc=3; nut=[(4,7,0,0),(2,7,1,4),(1,7,0,0),(3,9,2,0)]	ID=[3,2,1,4]; cac_muc=[[9],[7],[7,7]]; độ rộng=[1,1,2]; rộng nhất=2; mức=2; trung bình=[9.0,7.0,7.0].
T11.5 Input sai: gốc có cha và chu trình kiem_tra_dau_vao	goc=1; nut=[(1,10,2,0),(2,20,1,0)]	ArgumentError; nguyên nhân kiểm được đầu tiên là gốc có cha. Demo A08 bắt lỗi dự kiến và báo PASS vì đã từ chối input sai.

Bảng 4.4. Actual và vị trí minh chứng của 13 ca

ID	Actual và kết luận	Minh chứng
T01	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:10–23; scripts/test_catalog.jl:T01; A01; docs/logs/clone_C.log
T02	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:25–38; catalog T02; A02; clone_C.log
T03	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:40–53; catalog T03; A05; clone_C.log
T04	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:55–64; catalog T04; A07; clone_C.log
T05	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:66–77; catalog T05; A06; clone_C.log
T06	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:79–97; catalog T06; A03; clone_C.log
D04	Ảnh A04 in đúng các nhóm và mức 1; đối chiếu trực tiếp với sơ đồ cây. Kết luận: PASS.	demo.jl --ca=4; Hình 4.4; kết quả chạy demo trong log toàn bộ.
T07	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:99–109; catalog T07; clone_C.log
T08	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:111–122; catalog T08; clone_C.log
T09	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:124–143; catalog T09; clone_C.log
T10	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:145–160; catalog T10; clone_C.log

ID	Actual và kết luận	Minh chứng
T13	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:206–215; catalog T13; clone_C.log
T11.5	Khớp expected; các kiểm tra liên quan đạt PASS trong bộ test và bảng đối chiếu đã chạy. Kết luận: PASS.	test/runtests.jl:179–183; catalog T11.5; A08; clone_C.log

Mười hai dòng lấy từ catalog gồm T01–T10, T13 và input T11.5; dòng D04 bổ sung đúng cây năm nút dùng cho ảnh A04. D04 được đối chiếu từ ảnh và log demo, không được mô tả nhầm thành một assertion của T09 vì T09 dùng cây bảy nút và ba mức hòa. Tất cả 13 dòng có input hoặc mục tiêu riêng, còn các ảnh chỉ là bằng chứng của các ca đó.

T01–T06 bao gồm hai ví dụ trên phiếu và đủ năm ca bắt buộc; một nút vừa là ví dụ 2 vừa là ca bắt buộc nên chỉ tính một input tương ứng.

T07–T10 mở rộng sang giá trị chuỗi, nhánh xen kẽ, nhiều mức hòa và cây 15 nút. T13 kiểm gốc không phải 1, bản ghi xáo thứ tự và giá trị lặp.

T11 gồm bảy input validator. T15–T31, T36–T38 bổ sung giới hạn n, định dạng, kiểu dữ liệu, ID/con ngoài miền, chu trình rời và miền số.

T12, T33–T35 kiểm trung bình số, cây rỗng, tràn tổng số nguyên/số thực và mức chứa chuỗi. T32 kiểm kết quả giữa các lần gọi và việc giữ nguyên input.

I01–I16 kiểm luồng nhập/CLI, gồm từ chối n âm hoặc vượt 100 trước khi đọc nút, thiếu dòng, sai trường, sai kiểu, số thực, chuỗi, BigInt, chu trình và tham số sai.

Bảng catalog lấy lại 12 ca trong bộ T, gồm T01–T10, T13 và T11.5. Các ảnh A01–A08 là minh họa đã có; A09 chụp tổng kết 121/121 và 18/18, không tạo thêm số ca.

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Chạy julia --startup-file=no demo.jl --ca=1 với input ở Bảng 1.3. Kết quả thực tế là dãy 10→20→30→40 và nhóm [[10],[20,30],[40]], khớp kỳ vọng rộng nhất 2 tại mức

1. Hình 4.1 trình bày kết quả; Bảng 3.3–3.4 giải thích từng bước FIFO trên chính input này.

Việc 30 được lấy trước 40 là điểm phân biệt cần kiểm: nút 40 nằm sâu hơn dù được phát hiện sau khi xử lý 20. Bốn nút vào/ra queue đúng bốn lần; queue chờ lớn nhất bằng hai.

4.3.2. Ví dụ 2 — VD02

Chạy demo.jl --ca=2 với goc=1 và nut=[(1,8,0,0)]. Hình 4.2 cho một nhóm duy nhất [8], độ rộng 1 tại mức 0 và mỗi bộ đếm queue bằng một. So với ví dụ 1, không có nhánh thêm con; queue trở rỗng ngay sau lần lấy đầu.

Dạng in Any[8] hoặc Union{Nothing,Float64}[8.0] là thông tin kiểu của Julia; giá trị cần đối chiếu vẫn là nhóm [8] và trung bình [8.0]. Không coi tiền tố tên kiểu là một phần tử hay một mức bổ sung.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

TK01 dùng cây đầy bảy nút (T06, demo --ca=3). Từ sơ đồ cây, các mức có 1, 2, 4 nút; expected là rộng nhất 4 tại mức 2. Hình 4.3 xác nhận đúng các nhóm [[1],[2,3],[4,5,6,7]], giúp kiểm thứ tự trái trước phải ở nhiều cha cùng mức.

4.4.2. Ca TK02

TK02 dùng cây năm nút của demo --ca=4, với hai con của gốc và mỗi nhánh có một cháu. Mức 1 và mức 2 cùng rộng hai; expected phải giữ mức 1. Hình 4.4 in [1,2,2] và chọn 1, nhờ đó có thể phát hiện cài đặt dùng dấu ≥ để cập nhật rồi vô tình chọn mức cuối.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

BIEN01 là cây rỗng, đúng miền n=0 trên phiếu (T03, demo --ca=5). Hình 4.5 cho các dãy rỗng, rộng nhất 0, không có mức rộng nhất và không có thao tác queue. Nhánh này chứng minh chương trình xử lý trước khi truy cập ID gốc, không tạo mức 0 giả cho một cây không có nút.

4.5.2. Ca BIEN02

BIEN02 là dây phải bốn nút (T05, demo --ca=6), cũng là ca bắt buộc số 4. Hình 4.6 cho độ rộng [1,1,1,1] và mức 0. Mọi mức hòa bằng một; queue chờ tối đa một nút, nhưng hai mảng cấp phát vẫn có bốn ô mỗi mảng.

4.5.3. Ca BIEN03

BIEN03 là gốc có hai lá (T04, demo --ca=7), ca bắt buộc số 3 của phiếu. Hình 4.7 cho nhóm [[100],[200,300]], rộng nhất 2 tại mức 1. Đây là cấu hình nhỏ nhất làm độ rộng tăng từ gốc sang mức con, kiểm đồng thời việc thêm hai con và giữ thứ tự trái trước phải.

4.6. Kiểm tra input không hợp lệ — LOI01

LOI01 dùng goc=1; nut=[(1,10,2,0),(2,20,1,0)]. Liên kết 1→2→1 tạo chu trình và làm gốc có cha. Kỳ vọng là từ chối trước khi duyệt chính; thông báo thực tế trong Hình 4.8 là ArgumentError về bậc vào của gốc. Không yêu cầu phải đợi đến nhánh kiểm chu trình mới báo lỗi khi một vi phạm đã được phát hiện sớm hơn.

Demo A08 bắt đúng lỗi dự kiến rồi in PASS; trạng thái thành công ở đây nghĩa là phép kiểm đã từ chối input sai. Các input sai khác trong T11 và các nhóm hồi quy kiểm con chung, ID/con ngoài miền, dữ liệu rời và sai kiểu; không xem phản hồi từ chối là thuật toán chấp nhận cây đó.

4.7. Kết quả toàn bộ kiểm thử

Bảng 4.5. Tổng kết các bộ kiểm tra và cách tính

Bộ kiểm tra	**Nhóm hoặc đối tượng**	**Kết quả thực tế**
Thuật toán Julia	T01–T38, 38 nhóm ID; riêng T11 chứa bảy input validator	121/121 phép kiểm đạt; 0 phép kiểm không đạt trong lần chạy.
Đọc input và CLI Julia	I01–I16, 16 nhóm ID; I16 kiểm ba tham số sai	18/18 phép kiểm đạt; 0 phép kiểm không đạt.
Bảng catalog	12 ca đã có trong bộ T; không cộng thành ca mới	12/12 dòng đối chiếu PASS.
Giao diện HTML	24 ca logic thực thi script với mô hình DOM	24/24 ca đạt; không phải kiểm tra bố cục trình duyệt.

Bộ kiểm tra	Nhóm hoặc đối tượng	Kết quả thực tế
Demo và khảo sát	A01–A08, truy vết và sáu cấu hình tại 3/7/31 nút	Quy trình kết thúc thành công theo log thực chạy.

Hình 4.9 là ảnh cá nhân tổng kết hai bộ Julia, thể hiện 18/18 và 121/121. Tổng 139 là số phép kiểm Julia, không phải số cây độc lập. Số nhóm có ID, số input, số assertion và số ảnh là các đại lượng khác nhau, vì một input có thể kiểm nhiều thuộc tính và một nhóm có thể chứa nhiều input lỗi.

Không có lỗi kiểm thử còn mở trong những lần chạy được ghi. Tuy nhiên, tập kiểm thử hữu hạn không chứng minh chương trình đúng cho mọi giá trị có thể biểu diễn; phần kết luận tổng quát về duyệt mức dựa trên bất biến và tính chất cây ở Mục 3.5. Thời gian 2,x giây trên ảnh là thời gian bộ Test, không được dùng như benchmark của một lần duyệt cây.

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Đếm thao tác trên cây dây phải và cây đầy có cùng n=3,7,31; giá trị i*10, hai con cây đầy là 2i và 2i+1 nếu trong miền. Không đo thời gian.

Với mỗi n, cây dây có nút i trỏ con phải i+1 (nút cuối không có con). Cây đầy có con 2i và 2i+1 nếu còn trong miền; giá trị nút i là i×10. Dữ liệu được tạo bằng vòng lặp xác định, cùng dùng giai_bai và validator của chương trình, không sửa thuật toán theo từng dạng cây.

```
julia --startup-file=no scripts/trace_and_scales.jl --scales
```

4.8.2. Kết quả và giới hạn nhận xét

Bảng 4.6. Khảo sát Q1 Q2 Q3 trên hai dạng cây

Bộ / n	Dạng cây	Độ sâu lớn nhất	Vào / ra	qmax	Rộng nhất / mức
Q1 / 3	Dây phải	2	3 / 3	1	1 / 0
Q1 / 3	Cây đầy	1	3 / 3	2	2 / 1
Q2 / 7	Dây phải	6	7 / 7	1	1 / 0

Bộ / n	Dạng cây	Độ sâu lớn nhất	Vào / ra	qmax	Rộng nhất / mức
Q2 / 7	Cây đầy	2	7 / 7	4	4 / 2
Q3 / 31	Dây phải	30	31 / 31	1	1 / 0
Q3 / 31	Cây đầy	4	31 / 31	16	16 / 4

Số liệu được đối chiếu với output của scripts/trace_and_scales.jl trong docs/logs/clone_C.log và log toàn bộ. Cột độ sâu dùng số cạnh; nếu đổi sang chiều cao theo giáo trình, cộng một ở cây không rỗng. Đây là mô tả hình dạng cây, không phải đại lượng thay cho độ rộng.

Ở cả hai dạng, enqueue=n và dequeue=n, do đó tổng hai thao tác bằng 2n. Cùng phân tích mỗi bước O(1), kết quả hỗ trợ chi phí tuyến tính của lõi duyệt.

Cây dây luôn chỉ có một nút đang chờ; các cây đầy được chọn có 2,4,16 nút đang chờ lớn nhất. Khác biệt hình dạng ảnh hưởng queue hoạt động, dù mảng queue cấp phát sẵn n ô cho cả hai dạng.

Giới hạn khảo sát là ba kích thước nhỏ trong miền phiếu, dùng phép đếm thay đo thời gian và không đo dung lượng bộ nhớ theo byte. Kết quả hỗ trợ cách đếm 2n lượt queue chính; kết luận độ phức tạp toàn bộ vẫn dựa trên phân tích các pha. Không suy ra mọi cây cùng n có qmax giống nhau hoặc qmax luôn bằng độ rộng lớn nhất.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Commit C cố định mã/test/input dùng tạo kết quả và ảnh. Bản bổ sung ảnh được chốt sau đó; mã thực thi và tệp test đã được kiểm tra không đổi so với C. SHA S và xác nhận chạy lại được ghi trong TXT nộp rời theo quy định, tránh đưa hash của chính commit đang tạo vào nội dung thuộc commit đó.

Bảng 4.7. Kho mã và phiên bản đối chiếu

Thông tin	Giá trị
Repo GitHub thật	https://github.com/trucluong097-del/N24DCDT100_DT084_LuongMinhTruc

Thông tin	Giá trị
Full SHA C	ccaf13ab5c6bca2a9affc669f42508e7d0f6848a
Ngôn ngữ / runtime	Julia 1.12.6; Test và Printf chuẩn.
Giữa C và bản thêm ảnh	Mã .jl, .ps1, HTML và test GUI không đổi; bổ sung ảnh, danh mục, log và ZIP.
SHA S	Ghi trong TXT thông tin nộp rời sau khi chốt phiên bản cuối; báo cáo giữ SHA C.
Lệnh chạy tổng hợp	.\Run_all.ps1 hoặc chỉ định -Julia đường dẫn julia.exe.
Kiểm tra S	Đã clone bản bổ sung ảnh vào thư mục mới, đối chiếu 29 tệp và ZIP; hai ví dụ và hai bộ Julia chạy lại đạt 121/121 và 18/18.

Permalink thuật toán: src/DeTai.jl tại C

Permalink test thuật toán: test/runtests.jl tại C

Permalink test nhập liệu: test/input_tests.jl tại C

4.9.2. Clone sạch và tái hiện kết quả

Ảnh A10 cho thấy thư mục riêng C:\Users\TRUC\DT084_kiem_tra_C, URL origin đúng repo, SHA HEAD đúng C, Julia 1.12.6 và hai bộ test đạt. Ảnh không hiển thị chính lệnh clone; vì vậy chỉ dùng ảnh để xác nhận trạng thái phiên bản và kết quả chạy lại, kết hợp quy trình và nhật ký đã lưu, không mô tả thêm một thao tác không có trên ảnh.

```
git clone https://github.com/trucluong097-del/N24DCDT100_DT084_LuongMinhTruc.git
DT084_kiem_tra_C
cd DT084_kiem_tra_C
git checkout ccaf13ab5c6bca2a9affc669f42508e7d0f6848a
git rev-parse HEAD
julia --version
julia --startup-file=no demo.jl
julia --startup-file=no test/runtests.jl
julia --startup-file=no test/input_tests.jl
```

Nhật ký clone_C.log ghi output thực chạy demo, test, catalog, truy vết và khảo sát sau khi repo đã được clone; clone_C_gui.log ghi 24 ca logic HTML. Đây là nhật ký chạy chương trình, không phải ảnh hoặc bản ghi đầy đủ mọi thao tác tạo thư mục clone. Sau lần thêm ảnh, bản S đã được clone mới và chạy lại, đồng thời so sánh nội dung với ZIP để kiểm tính nhất quán.

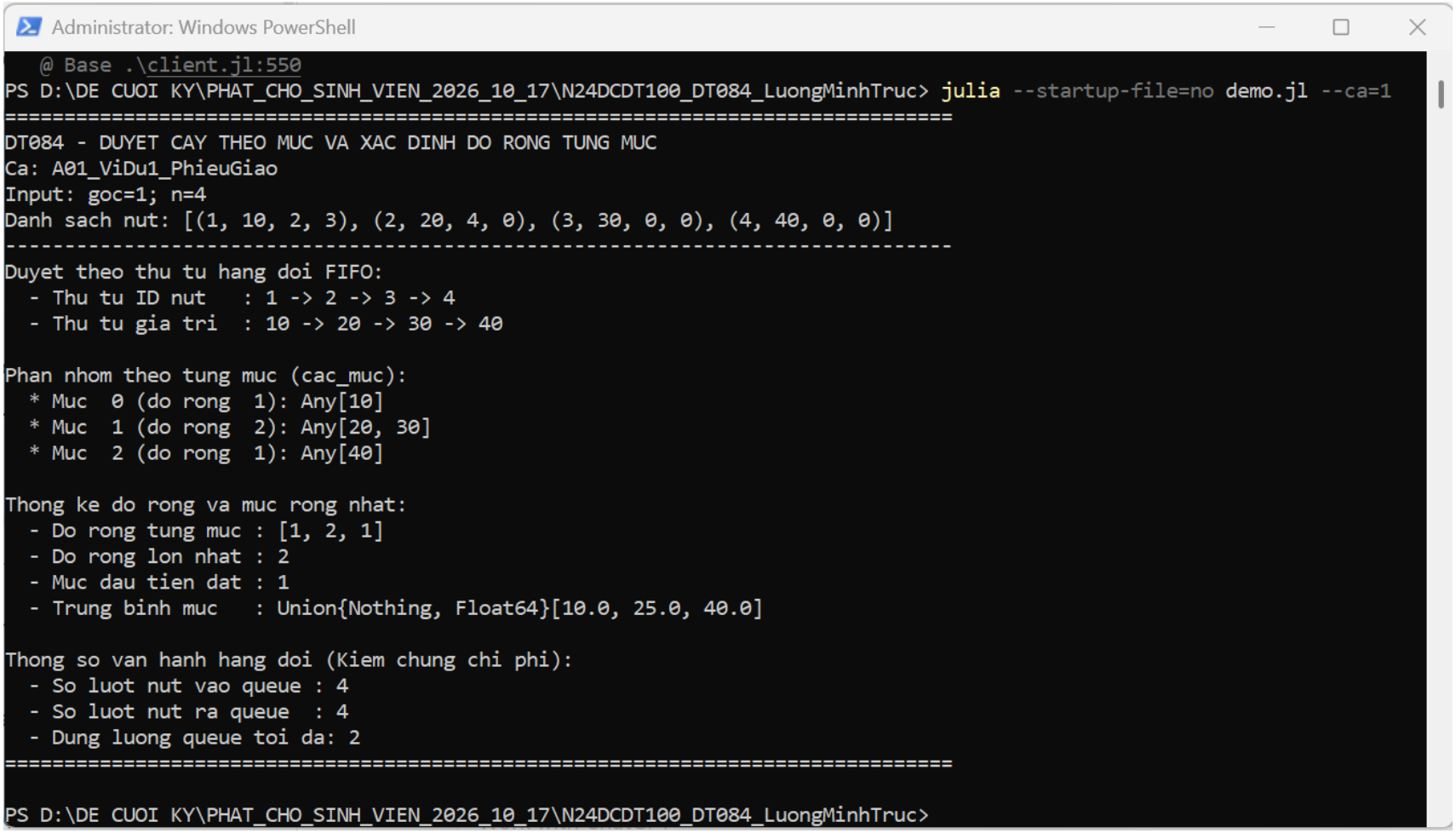
4.9.3. Bằng chứng kho GitHub thật

Hình 4.11 ghi tài khoản trucluong097-del, repo N24DCDT100_DT084_LuongMinhTruc, commit ccaf13a và 17 tệp thay đổi. SHA rút gọn trên trang đối chiếu với SHA đầy đủ trong A10 và Bảng 4.7. Repo công khai cho phép mở mã và permalink; báo cáo không coi trạng thái công bố mã là kết quả chấm hoặc kết quả bảo vệ.

4.10. Bộ ảnh minh chứng cá nhân

Mười một ảnh gốc A01.png–A11.png được giữ trong docs/anh_ket_qua và ZIP mã nguồn. Các trang ảnh sau dùng A4 ngang để chữ terminal không bị thu quá nhỏ; không sửa nội dung kết quả trong ảnh. Lệnh, input, expected và actual đã được mô tả ở các mục 4.3–4.9 và các bảng đối chiếu; dưới mỗi hình có ít nhất hai câu phân tích.

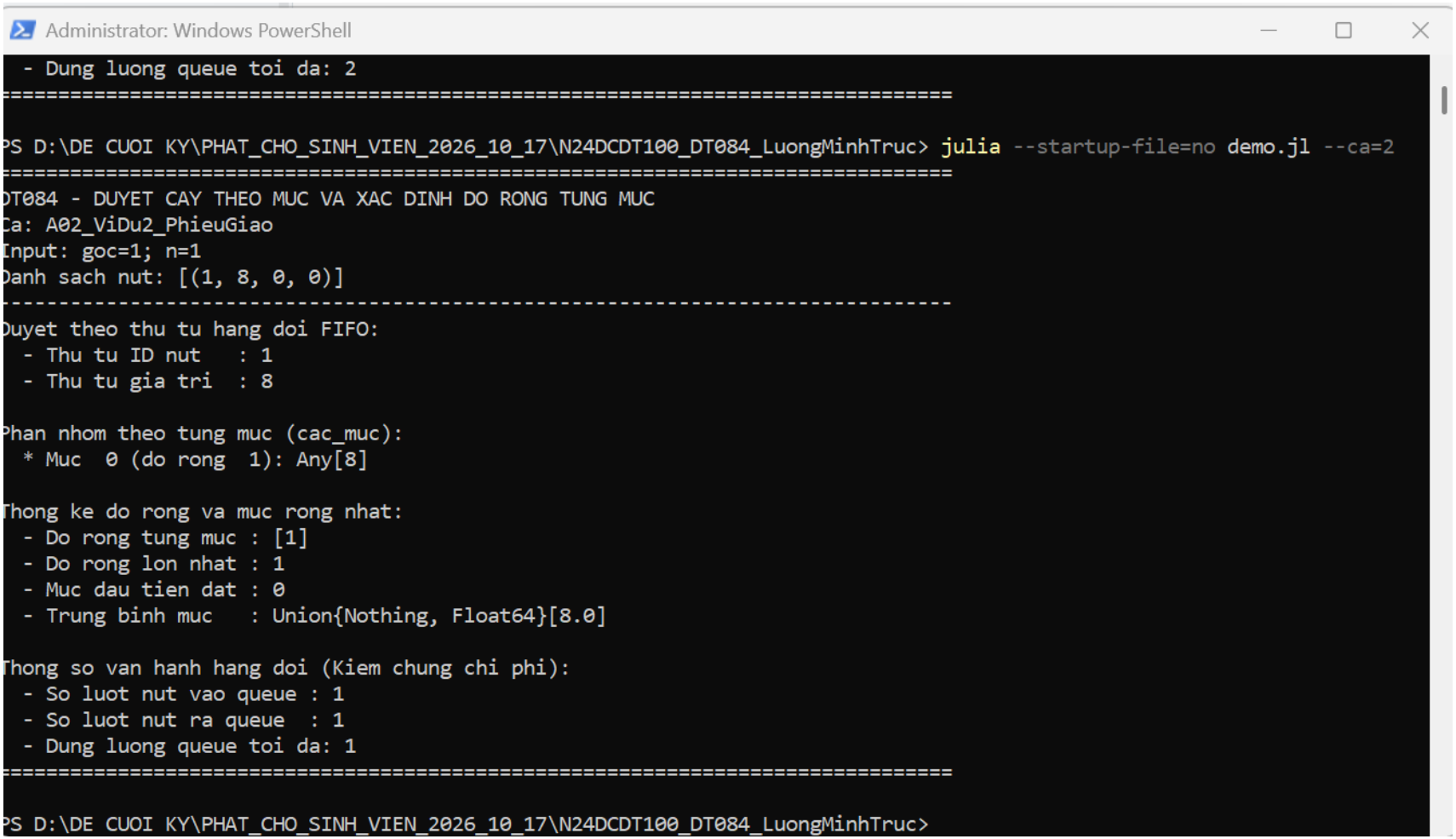
A01 Ví dụ 1 trên phiếu



Hình 4.1. Ví dụ 1 trên phiếu

Hai nút 20 và 30 cùng mức được xử lý trước cháu 40. Các nhóm [[10],[20,30],[40]] cho độ rộng [1,2,1], đúng kỳ vọng rộng nhất 2 tại mức 1.

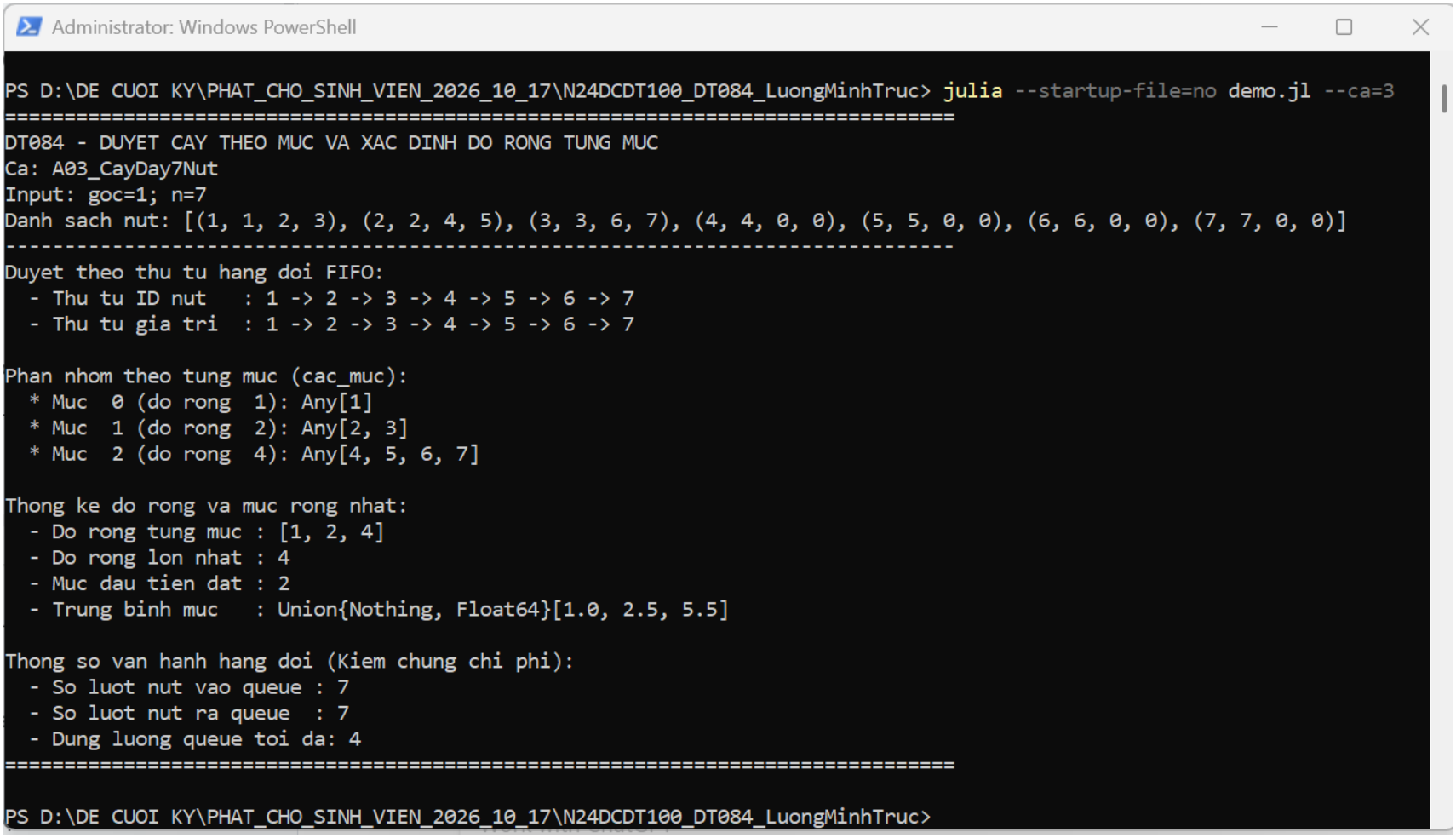
A02 Ví dụ 2 trên phiếu



Hình 4.2. Ví dụ 2 trên phiếu

Cây chỉ có gốc giá trị 8 nên có một mức duy nhất. Kết quả rộng nhất 1 tại mức 0 và một lượt vào/ra queue khớp kỳ vọng.

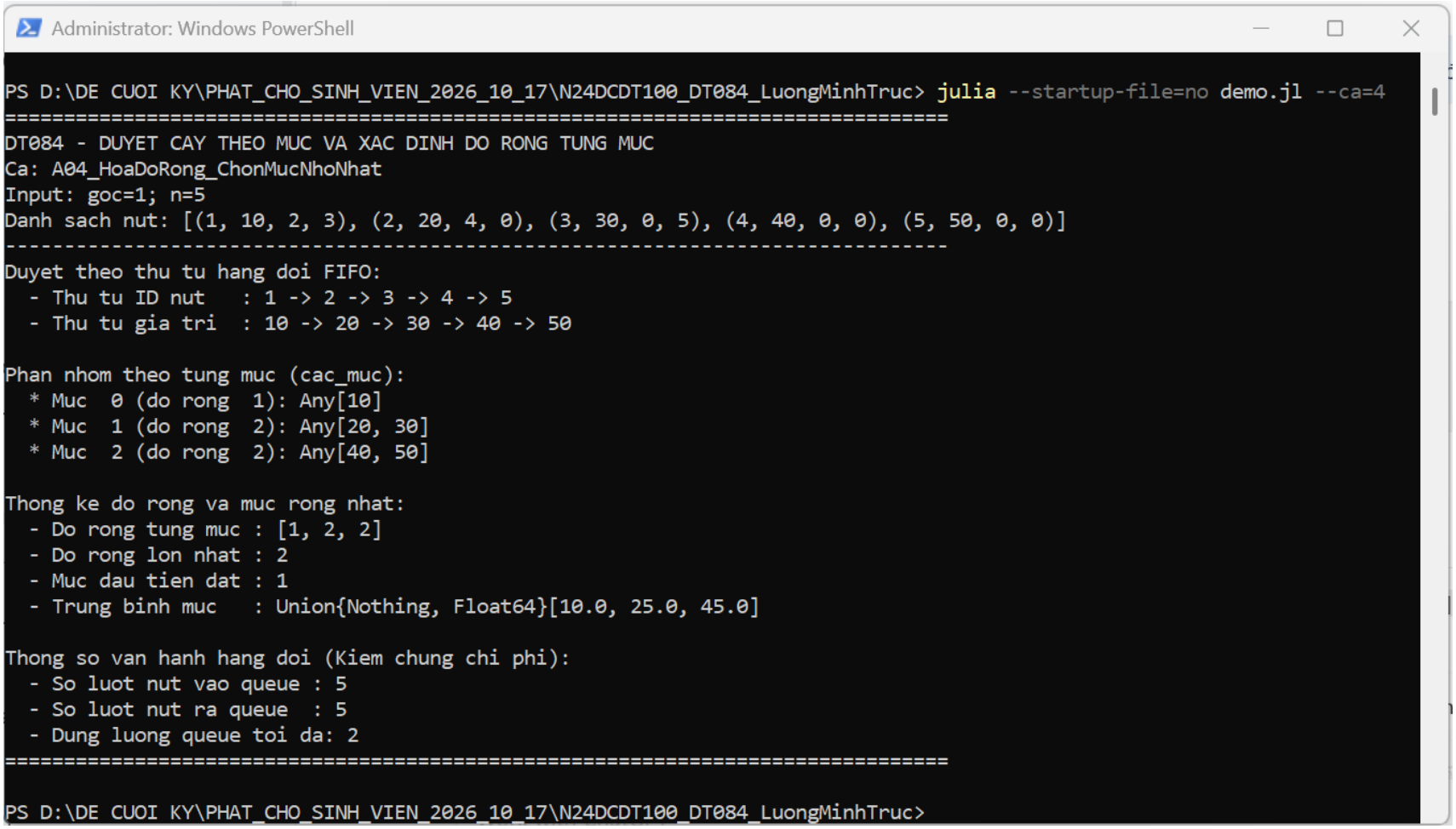
A03 Ca tự thiết kế cây đầy bảy nút



Hình 4.3. Ca tự thiết kế cây đầy bảy nút

Các nhóm có lần lượt 1, 2 và 4 nút, đồng thời giữ thứ tự trái trước phải. Bốn nút lá ở mức 2 tạo độ rộng lớn nhất 4, đúng sơ đồ cây.

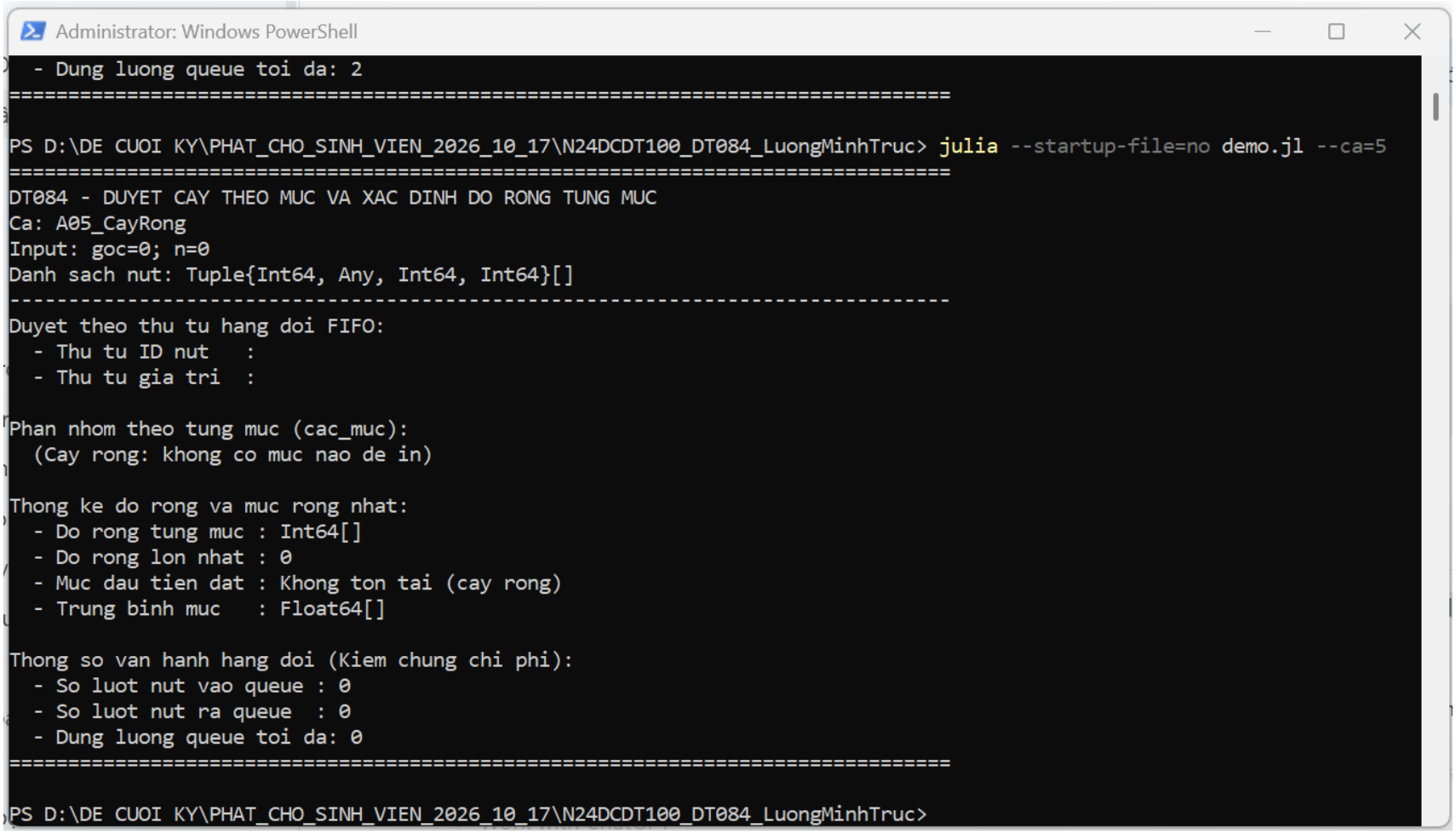
A04 Ca tự thiết kế hòa độ rộng



Hình 4.4. Ca tự thiết kế hòa độ rộng

Mức 1 và mức 2 cùng có hai nút. Chương trình giữ mức 1 là mức đầu tiên đạt rộng nhất, đúng quy tắc xử lý hòa.

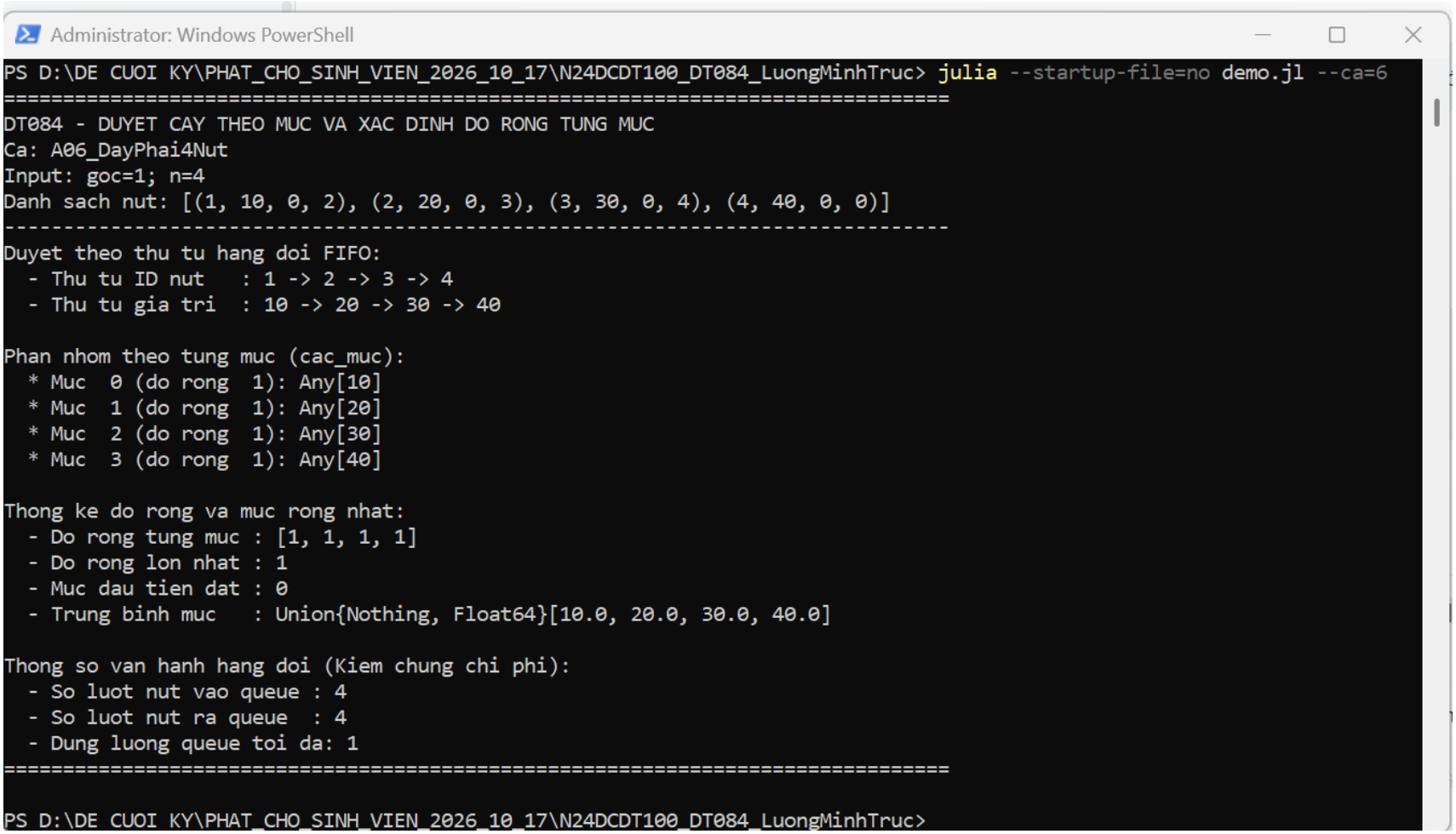
A05 Ca biên cây rỗng



Hình 4.5. Ca biên cây rỗng

Không có nút nên không tạo nhóm mức hoặc thêm gốc giả vào queue. Kết quả rộng nhất 0, mức không tồn tại và các bộ đếm bằng 0 khớp quy ước.

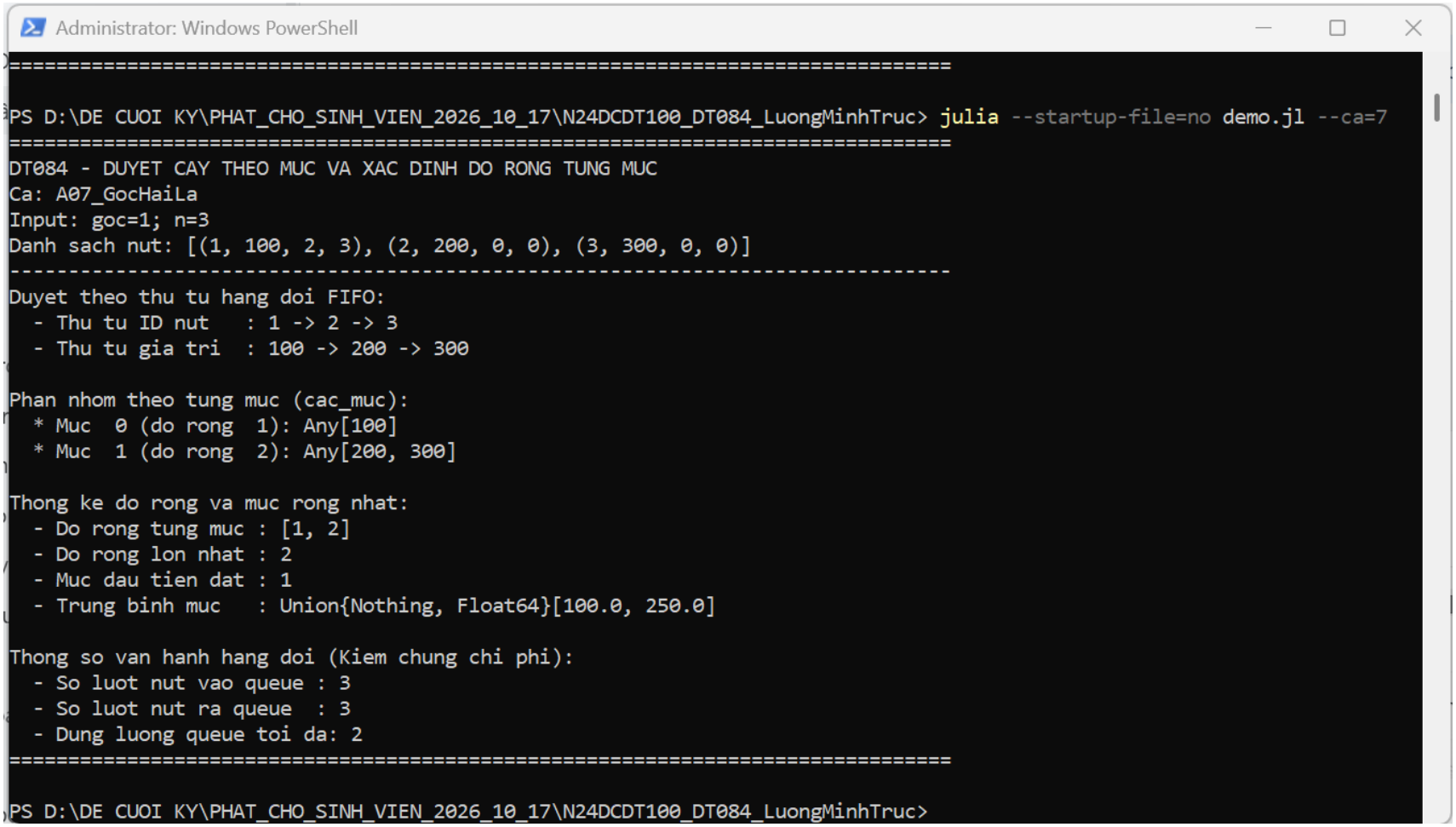
A06 Ca biên dây phải bốn nút



Hình 4.6. Ca biên dây phải bốn nút

Mỗi mức có một nút nên độ rộng là [1,1,1,1]. Kết quả chọn mức 0 và tối đa một nút đang chờ phù hợp cây dây; mảng queue vẫn cấp phát theo n.

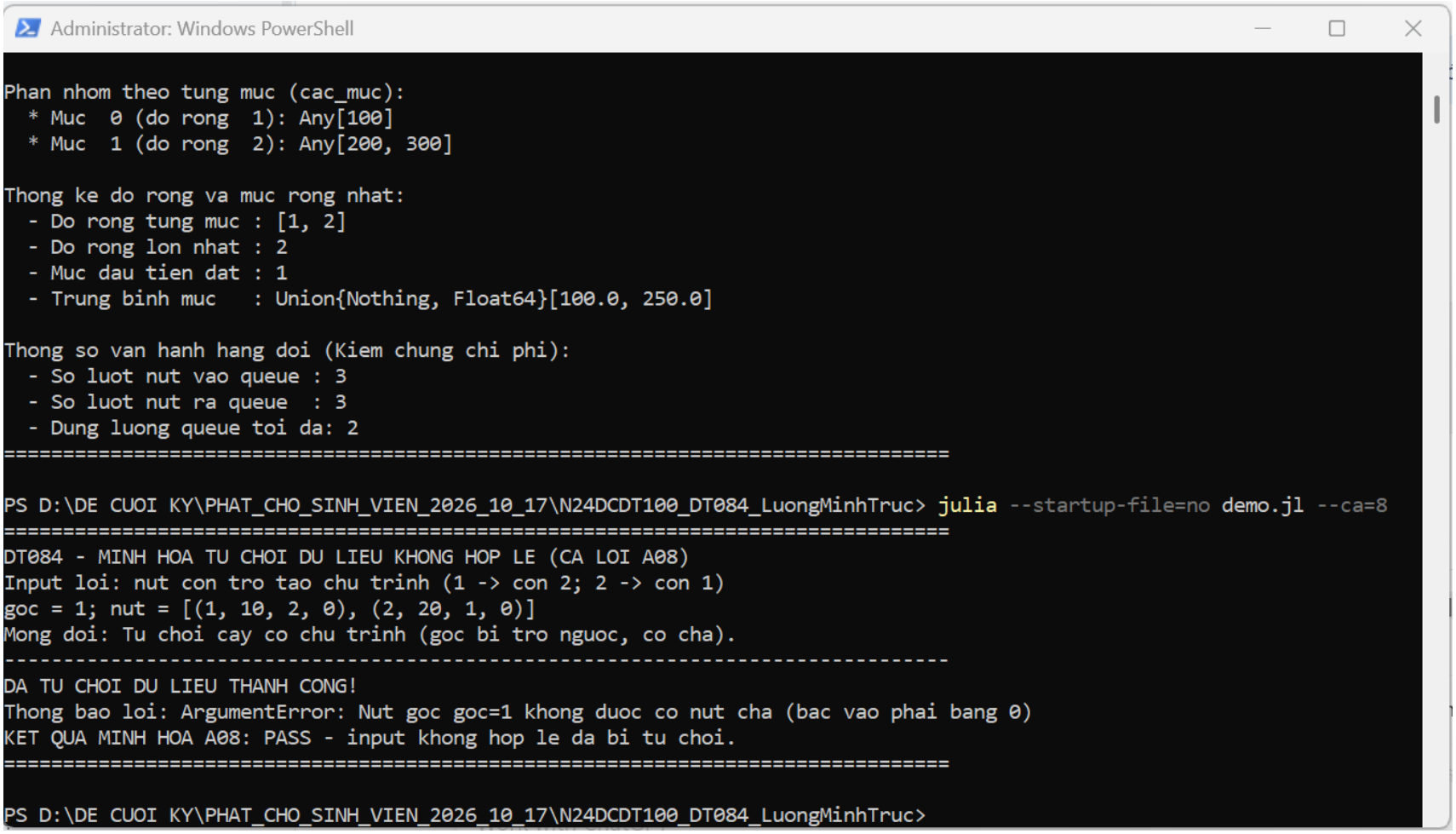
A07 Ca biên gốc có hai lá



Hình 4.7. Ca biên gốc có hai lá

Hai con của gốc nằm cùng mức 1, theo thứ tự trái trước phải. Độ rộng [1,2] cho rộng nhất 2 tại mức 1, đúng kỳ vọng.

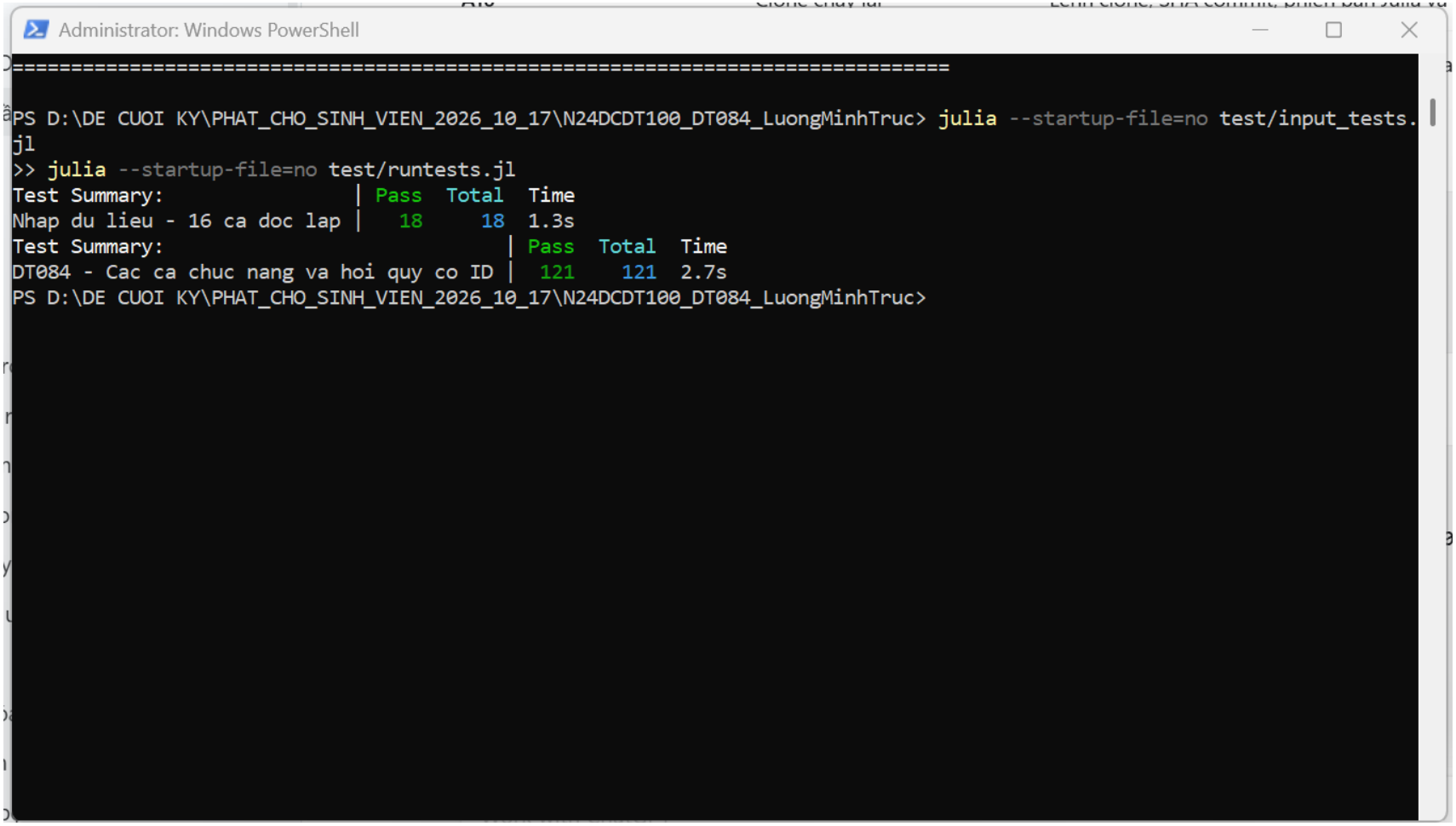
A08 Từ chối input không hợp lệ



Hình 4.8. Từ chối input không hợp lệ

Liên kết 1→2→1 làm gốc có cha và tạo chu trình. Validator báo ArgumentError trước duyệt chính; PASS nghĩa là từ chối dữ liệu sai đúng thiết kế.

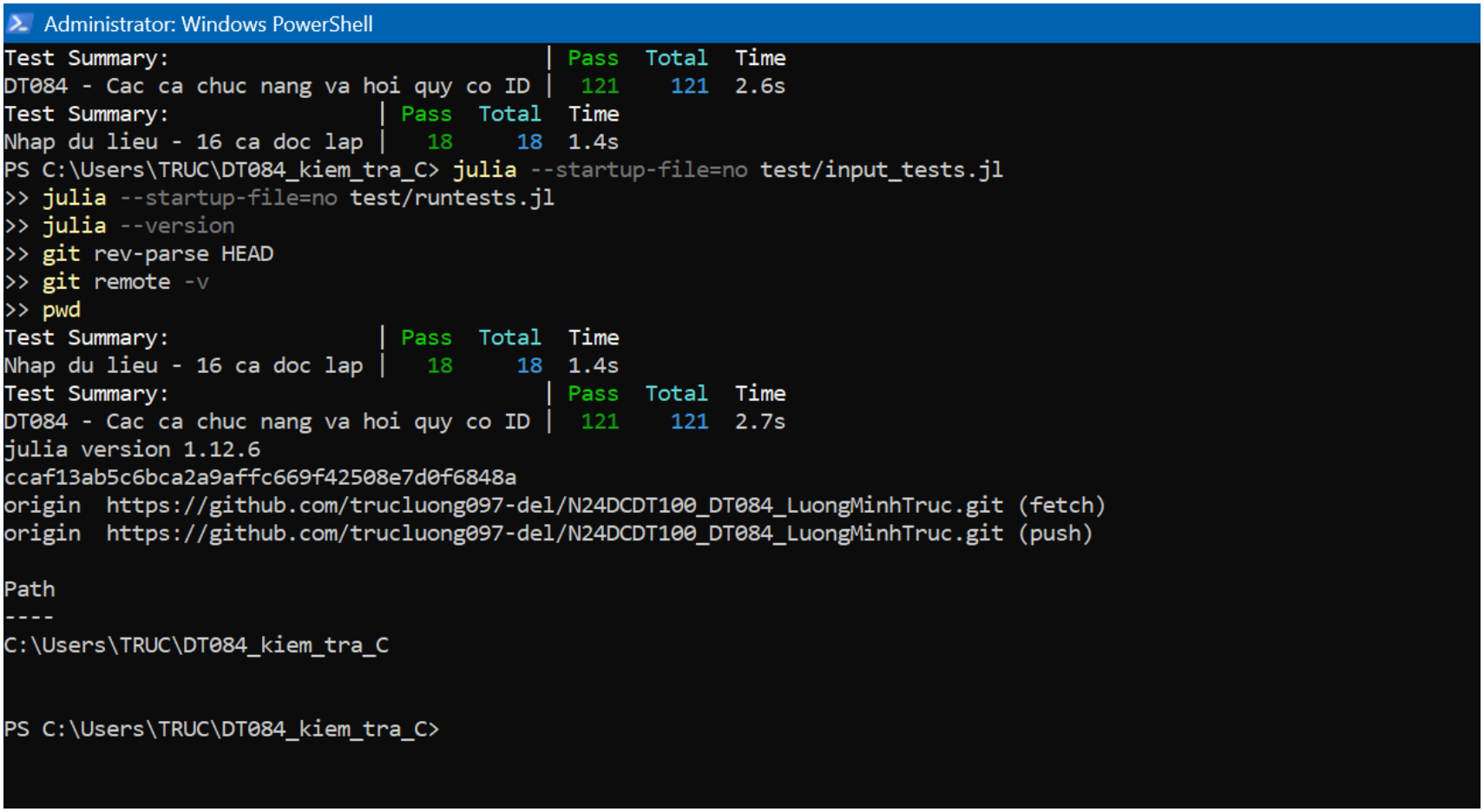
A09 Tổng kết bộ kiểm thử Julia



Hình 4.9. Tổng kết bộ kiểm thử Julia

Bộ nhập liệu đạt 18/18 phép kiểm và bộ thuật toán đạt 121/121. Tổng 139 là số phép kiểm, không phải số ca độc lập; danh mục nhóm và input nằm trong các tệp test.

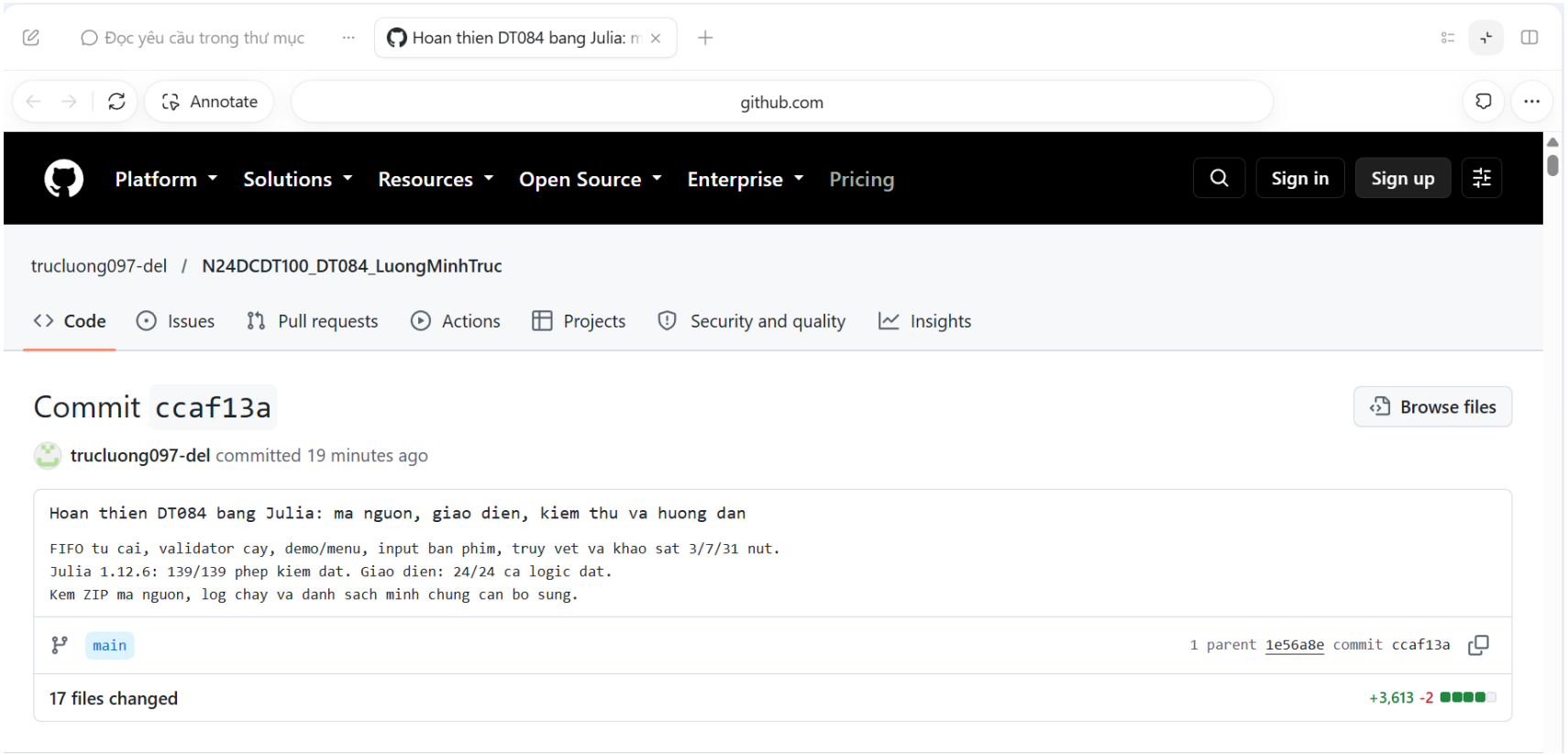
A10 Kiểm phiên bản C và chạy lại tại thư mục clone



Hình 4.10. Kiểm phiên bản C và chạy lại tại thư mục clone

Origin, SHA HEAD và Julia 1.12.6 xác định đúng mã đã chạy trong thư mục kiểm tra riêng. Hai bộ test vẫn đạt 121/121 và 18/18; ảnh không hiển thị chính lệnh clone.

A11 Repo và commit GitHub thật



Hình 4.11. Repo và commit GitHub thật

Trang GitHub hiển thị tài khoản, repo và commit C ccaf13a với 17 tệp thay đổi. Mã commit khớp ảnh A10, cho phép đối chiếu nguồn công bố với kết quả chạy.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Chương trình đã thực hiện các yêu cầu bắt buộc của DT084: biểu diễn cây bằng ID, tự cài FIFO để duyệt theo mức, giữ trái trước phải, đếm độ rộng và chọn mức đầu tiên khi hòa. Hai ví dụ và đủ năm ca trên phiếu đều có kết quả đối chiếu; cây rỗng và input sai được xử lý rõ ràng. Phần tính trung bình từng mức đã cài theo yêu cầu mở rộng tùy chọn.

Bằng chứng thực nghiệm gồm 121 phép kiểm thuật toán và 18 phép kiểm nhập liệu đạt, 24 ca logic giao diện đạt, bảng truy vết và khảo sát hai dạng cây tại ba quy mô hợp lệ. Mười một ảnh cá nhân, nguồn tại C và chạy lại từ clone giúp xác minh sản phẩm. Các kết quả này thể hiện mức hoàn thành kỹ thuật trong phạm vi đã kiểm, không tự xác định điểm học phần hoặc kết quả bảo vệ.

Bảng 5.1. Mức hoàn thành và bằng chứng

Yêu cầu	**Mức thực hiện**	**Bằng chứng và giới hạn**
Đặc tả đúng phiếu	Đã thực hiện	Phiếu nguyên bản; Bảng 1.2–1.3; miền n=0..100 và mức gốc 0.
Tự cài FIFO và duyệt	Đã thực hiện	src/DeTai.jl; Mục 3.3–3.5; dùng mảng/hai chỉ số, không dùng hàm duyệt có sẵn.
Độ rộng, hòa và cây rỗng	Đã thực hiện	A01–A07; nhóm T01–T10 và các phép kiểm tương ứng.
Kiểm input và nhập liệu	Đã thực hiện	A08, T11 và hồi quy; I01–I16. Giới hạn giá trị và chuỗi được ghi rõ.
Kiểm thử và khảo sát	Đã thực hiện trong tập đã nêu	13 ca đối chiếu, 139 phép kiểm Julia; Q1–Q3; không tuyên bố benchmark hay kiểm vét cạn.
Ảnh và khả năng tái hiện	Đã có bằng chứng	11 ảnh, repo/permalink C, clone và test S. SHA S quản lý trong TXT nộp rời.
Trung bình và HTML	Đã có phần mở rộng	Trung bình Float64 gần đúng; HTML mô phỏng độc lập, không phải thực thi Julia.

Yêu cầu	Mức thực hiện	Bằng chứng và giới hạn
Bảo vệ trực tiếp	Phần chuẩn bị, chưa có kết quả chấm	Có menu, nhập input mới và nội dung giải thích; giảng viên đánh giá khi bảo vệ.

5.2. Giới hạn và bài học

Trung bình dùng Float64 nên có làm tròn; việc tránh tràn tổng không đồng nghĩa tính tổng chính xác tùy ý.

Giá trị chuỗi được duyệt và in bình thường; mức có chuỗi không có trung bình số và trả nothing.

Nhập bàn phím dùng một token cho giá trị mỗi nút, nên chuỗi có khoảng trắng cần khai báo trực tiếp bằng mã hoặc dùng giao diện HTML theo quy tắc riêng.

Giao diện HTML là mô phỏng JavaScript độc lập, không được dùng ảnh HTML để khẳng định đã thực thi mã Julia.

Bộ test đã đạt là tập hữu hạn; không tuyên bố vét cạn toàn bộ cây nhị phân có tối đa 100 nút.

Lỗi n âm ở tầng nhập đã được sửa: dòng -1 0 từng có thể bị hiểu thành cây rỗng do bỏ qua nhập nút. Bản hiện tại kiểm miền n ngay sau dòng đầu; I01–I02 xác nhận từ chối n âm/vượt 100 trước khi đọc nút. Điều này cho thấy cần kiểm hợp đồng ở đúng tầng còn giữ thông tin input ban đầu.

Lỗi tràn tổng nguyên khi tính trung bình được sửa bằng chuyển Float64 và chia trước khi cộng, kèm các ca T33–T34. Với HTML, cây được kiểm trước khi đổi trạng thái để tránh chu trình gây lỗi khi vẽ; kết quả mô phỏng vẫn được phân biệt với việc thực thi Julia.

5.3. Hướng phát triển phù hợp

Có thể bổ sung bộ sinh cây xác định và cách đối chiếu mức độc lập, lưu seed/input để tái hiện lỗi. Hướng khác là thống nhất nhập chuỗi nhiều từ giữa bàn phím và HTML, rồi cân nhắc kết nối HTML với Julia qua dịch vụ cục bộ. Các hướng này chưa triển khai và không được cộng vào kết quả kiểm thử đã báo cáo.

TÀI LIỆU THAM KHẢO

[1] Học viện Công nghệ Bưu chính Viễn thông. Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf trong gói học phần. Tham chiếu: mục 5.4, trang in 130–132 (PDF 142–144); Chương 7, trang in 161–165 (PDF 173–177), đặc biệt mục 7.3.2 trang in 165 (PDF 177). Bản tệp được cung cấp không xác định rõ tác giả cá nhân/năm xuất bản trên bìa; báo cáo không suy từ ngày tạo PDF.

[2] Hồ Nhựt Minh, bộ phiếu giao đề tài INT13185, Nhóm 03, phát hành cùng gói ngày 17/10/2026; phiếu DT084 của sinh viên N24DCDT100. Kèm 00_HUONG_DAN_SINH_VIEN.html, rubric chung và khung báo cáo cá nhân. Dùng để xác định miền dữ liệu, hai ví dụ, năm ca bắt buộc và yêu cầu minh chứng.

[3] Lương Minh Trực, mã nguồn DT084 tại commit C ccaf13ab5c6bca2a9affc669f42508e7d0f6848a. Repo và mã có thể kiểm lại

Các tệp được dùng trong báo cáo: src/DeTai.jl; demo.jl; menu.jl; test/runtests.jl; test/input_tests.jl; scripts/test_catalog.jl; scripts/trace_and_scales.jl; test/gui_tests.cjs. Truy cập và đối chiếu ngày 28/10/2026.

[4] Bộ ảnh và nhật ký cá nhân DT084. Thư mục minh chứng trong repo

Nguồn này gồm A01–A11, danh mục ảnh và các log chạy thật; dùng cho các kết quả ở Chương 4. Công cụ ChatGPT/Codex được sử dụng để hỗ trợ hoàn thiện mã/kiểm thử và biên tập, như đã khai báo trong Lời cam đoan; không dùng như nguồn thay thế giáo trình hoặc bằng chứng đã tự chạy.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

```
N24DCDT100_DT084_LuongMinhTruc/
  README.md
  Run_all.ps1
  demo.jl
  menu.jl
  Giao_dien_truc_quan.html
  src/DeTai.jl
  test/runtests.jl
  test/input_tests.jl
  test/gui_tests.cjs
  scripts/test_catalog.jl
  scripts/trace_and_scales.jl
  docs/anh_ket_qua/A01.png ... A11.png
  docs/anh_ket_qua/README.md
  docs/logs/
  docs/TINH_TRANG_MINH_CHUNG.md
```

Chạy PowerShell tại thư mục dự án, không nhập các lệnh shell dưới đây vào dấu nhắc julia>. Run_all.ps1 có thể tìm Julia trên PATH hoặc thư mục Juliaup; nếu cần chỉ định đường dẫn thực tế bằng tham số -Julia. Chương trình không yêu cầu cài gói ngoài.

```
.\Run_all.ps1
julia --startup-file=no menu.jl
julia --startup-file=no demo.jl --ca=1
julia --startup-file=no demo.jl --ca=2
julia --startup-file=no demo.jl --nhap
julia --startup-file=no test/runtests.jl
julia --startup-file=no test/input_tests.jl
julia --startup-file=no scripts/test_catalog.jl
julia --startup-file=no scripts/trace_and_scales.jl
```

Input bàn phím cho ví dụ 1:

```
4 1
1 10 2 3
2 20 4 0
3 30 0 0
4 40 0 0
```

Input cây rỗng: 0 0. Để kiểm logic HTML, có thể chạy node test/gui_tests.cjs; Node chỉ là công cụ kiểm tra giao diện tùy chọn, không phải phụ thuộc của chương trình Julia. Mở HTML bằng trình duyệt để mô phỏng, không chạy tệp HTML bằng Julia.

PHỤ LỤC B. Danh mục bằng chứng

Bảng PL.1. Liên hệ ảnh với ca kiểm và kết quả

Ảnh	**Ca / nội dung**	**Tệp và đối chiếu**
A01	Ví dụ 1 trên phiếu	docs/anh_ket_qua/A01.png; Hình 4.1; phiên bản C đã ghi ở Bảng 4.7.
A02	Ví dụ 2 trên phiếu	docs/anh_ket_qua/A02.png; Hình 4.2; phiên bản C đã ghi ở Bảng 4.7.
A03	Ca tự thiết kế cây đầy bảy nút	docs/anh_ket_qua/A03.png; Hình 4.3; phiên bản C đã ghi ở Bảng 4.7.
A04	Ca tự thiết kế hòa độ rộng	docs/anh_ket_qua/A04.png; Hình 4.4; phiên bản C đã ghi ở Bảng 4.7.
A05	Ca biên cây rỗng	docs/anh_ket_qua/A05.png; Hình 4.5; phiên bản C đã ghi ở Bảng 4.7.
A06	Ca biên dây phải bốn nút	docs/anh_ket_qua/A06.png; Hình 4.6; phiên bản C đã ghi ở Bảng 4.7.
A07	Ca biên gốc có hai lá	docs/anh_ket_qua/A07.png; Hình 4.7; phiên bản C đã ghi ở Bảng 4.7.
A08	Từ chối input không hợp lệ	docs/anh_ket_qua/A08.png; Hình 4.8; phiên bản C đã ghi ở Bảng 4.7.
A09	Tổng kết bộ kiểm thử Julia	docs/anh_ket_qua/A09.png; Hình 4.9; phiên bản C đã ghi ở Bảng 4.7.
A10	Kiểm phiên bản C và chạy lại tại thư mục clone	docs/anh_ket_qua/A10.png; Hình 4.10; phiên bản C đã ghi ở Bảng 4.7.
A11	Repo và commit GitHub thật	docs/anh_ket_qua/A11.png; Hình 4.11; phiên bản C đã ghi ở Bảng 4.7.

Danh mục chi tiết, input và hai câu phân tích từng ảnh còn có trong docs/anh_ket_qua/README.md. Các log clone_C.log, clone_C_gui.log và kiem_tra_toan_bo.log bổ sung output dài. Ảnh A10 xác minh trạng thái thư mục clone/HEAD và test, không phải ảnh hiển thị lệnh clone.

PHỤ LỤC C. Tự kiểm và danh mục kiểm thử

Bảng PL.2. Danh mục 38 nhóm kiểm thử thuật toán

ID	Mục tiêu của nhóm
T01	Ví dụ 1 bốn nút
T02	Một nút
T03	Cây rỗng
T04	Gốc và hai lá
T05	Dây phải bốn nút
T06	Cây đầy bảy nút
T07	Dây trái, chuỗi
T08	Zig-zag
T09	Hòa ba mức
T10	Cây đầy mười lăm nút
T11	Bảy input validator
T12	Trung bình và cây rỗng
T13	Gốc khác 1/bản ghi đảo/giá trị lặp
T14	n=100 hợp lệ
T15	n=101 bị từ chối
T16	ID trùng
T17	Con ngoài miền
T18	Gốc không nguyên
T19	Danh sách sai kiểu
T20	Bản ghi vô hướng
T21	Thiếu trường
T22	Thừa trường
T23	ID số thực

ID	Mục tiêu của nhóm
T24	Con chuỗi
T25	BigInt ID quá lớn
T26	BigInt con quá lớn
T27	NaN
T28	Vô cực
T29	Số phức
T30	Bool dùng làm ID
T31	Giá trị vector
T32	Trạng thái giữa các lần gọi
T33	Tràn tổng nguyên
T34	Tràn tổng thực
T35	Mức chứa chuỗi
T36	Chu trình rời gốc
T37	Số hữu hạn quá miền Float64
T38	Bool giá trị và gốc

Bảng PL.3. Danh mục 16 nhóm kiểm thử nhập liệu

ID	Input hoặc tình huống	Expected
I01	-1 0\n	ArgumentError trước khi đọc nút.
I02	101 1\n	ArgumentError trước khi đọc nút.
I03	0 0\n	goc=0, danh sách rỗng.
I04	0 1\n	ArgumentError.
I05	2 1\n1 10 2 0\n	ArgumentError vì thiếu dòng thứ hai.
I06	1 1 7\n	ArgumentError vì header thừa trường.
I07	1.5 1\n	ArgumentError vì n không nguyên.

ID	Input hoặc tình huống	Expected
I08	1 1\n1 9 0\n	ArgumentError vì nút thiếu trường.
I09	1 1\n1.2 9 0 0\n	ArgumentError vì ID không nguyên.
I10	1 1\n1 2.5 0 0\n	Trung bình [2.5].
I11	1 1\n1 Hoa 0 0\n	Dãy ["Hoa"].
I12	1 1\n1 Inf 0 0\n	ArgumentError.
I13	3 2\n3 30 0 0\n2 20 1 3\n1 10 0 0\n	Dãy [20,10,30].
I14	2 1\n1 10 2 0\n2 20 1 0\n	ArgumentError.
I15	1 1\n1 9223372036854775808 0 0\n	Giá trị BigInt 9223372036854775808 được giữ đúng.
I16	main_demo(["--ca=abc"]); main_demo(["--nhap","--ca=1"]); main_demo(["--ca=9"])	Ba lời gọi đều ArgumentError.

Bảng PL.4. Tự kiểm sản phẩm trước khi nộp

Nội dung	Đối chiếu
Thông tin cá nhân và mã đề	Đã đối chiếu phiếu DT084 / N24DCDT100 / Lương Minh Trực / D24CQDT01-N.
Cài đặt và nguồn hỗ trợ	Đã mô tả phần FIFO/validator tự cài, thư viện chuẩn và hỗ trợ ChatGPT/Codex.
Ca, ảnh và khảo sát	Có ít nhất 12 ca độc lập; 11 ảnh thật; truy vết; tính đúng; 3 quy mô hợp lệ.
Nguồn có thể chạy lại	Repo, SHA C và permalink đã có; clone C/S và test đã được kiểm tra.
Báo cáo	Đã hoàn thiện nội dung năm chương, chèn ảnh và cập nhật các danh mục. Phiếu điểm và chữ ký để đúng người điền.
Hồ sơ Classroom	Nộp Word, PDF, ZIP và TXT cùng phiên bản. Hạn nộp, lịch bảo vệ và thao tác Nộp bài theo Classroom; báo cáo không xác nhận thay trạng thái nộp thực tế.

PHỤ LỤC D. Hướng dẫn nộp Classroom

Tiền tố thống nhất là N24DCDT100_DT084_LuongMinhTruc. Dùng tên không dấu, không khoảng trắng, giữ đúng phần mở rộng. Hạn nộp và lịch bảo vệ lấy từ thông báo của giảng viên trên Classroom; ngày hoàn thiện báo cáo không thay cho hạn nộp.

Bảng PL.5. Bốn tệp theo quy định học phần

Tệp	**Tên và nội dung**
Word	N24DCDT100_DT084_LuongMinhTruc_BaoCao.docx — báo cáo đã điền; cập nhật mục lục, danh sách bảng/hình và số trang.
PDF	N24DCDT100_DT084_LuongMinhTruc_BaoCao.pdf — xuất từ đúng bản Word cuối và kiểm tra từng trang.
Mã nguồn	N24DCDT100_DT084_LuongMinhTruc_MaNguon.zip — code, demo/nhập, test, README, ảnh gốc và log; không có .git, cache hoặc bộ cài.
Thông tin nộp	N24DCDT100_DT084_LuongMinhTruc_ThongTinNop.txt — URL, SHA C/S, permalink, môi trường, lệnh và kết quả chạy lại; nộp rời.

Các thông tin cố định để điền TXT: URL repo ở Bảng 4.7; SHA C là ccaf13ab5c6bca2a9affc669f42508e7d0f6848a; môi trường Julia 1.12.6 trên Windows; test thuật toán 121/121 và nhập liệu 18/18. SHA S phải lấy ở phiên bản nộp cuối, sau khi chốt các tệp được đưa lên repo; nếu tiếp tục sửa mã/test/input, chạy lại phần bị ảnh hưởng và cập nhật bằng chứng.

Nếu chỉ bổ sung ảnh hoặc báo cáo mà mã/test/input không đổi so với C, giữ SHA C và ảnh tại C, rồi ghi SHA S và xác nhận clone S trong TXT rời. Không tạo vòng lặp commit chỉ để ghi hash của chính commit vào tệp thuộc commit đó. Đính kèm bốn tệp vào đúng bài tập Classroom, mở kiểm tra và bấm Nộp bài; xác nhận trạng thái đã nộp trên hệ thống. Ảnh gốc đã nằm trong báo cáo và ZIP nên không cần nộp riêng 11 ảnh.